
Accelerating Natural Gradient Descent for PINNs with Randomized Numerical Linear Algebra

Ivan Bioli

Department of Mathematics
Department of Civil Engineering and Architecture
University of Pavia
27100 Pavia, Italy
ivan.bioli@unipv.it

Carlo Marcati

Department of Mathematics
University of Pavia
27100 Pavia, Italy
carlo.marcati@unipv.it

Giancarlo Sangalli

Department of Mathematics
University of Pavia
IMATI-CNR
27100 Pavia, Italy
giancarlo.sangalli@unipv.it

Abstract

Natural Gradient Descent (NGD) has emerged as a promising optimization algorithm for training neural network-based solvers for partial differential equations (PDEs), such as Physics-Informed Neural Networks (PINNs). However, its practical use is often limited by the high computational cost of solving linear systems involving the Gramian matrix. While matrix-free NGD methods based on the conjugate gradient (CG) method avoid explicit matrix inversion, the ill-conditioning of the Gramian significantly slows the convergence of CG. In this work, we extend matrix-free NGD to broader classes of problems than previously considered and propose the use of Randomized Nyström preconditioning to accelerate convergence of the inner CG solver. The resulting algorithm demonstrates substantial performance improvements over existing NGD-based methods on a range of PDE problems discretized using neural networks.

1 Introduction

Neural network-based approaches for solving partial differential equations (PDEs) have gained significant attention as promising alternatives to traditional numerical methods for both forward and inverse problems. A common strategy is to embed the governing PDE directly into the loss function, as in Physics-Informed Neural Networks (PINNs) [24, 59, 63], the Deep Ritz method [25], and related variants [7, 8, 22, 32, 41]. These methods hold the promise of providing mesh-free approximations of PDE solutions and of exploiting the dimension-independent approximation power of neural networks [16]. However, their practical adoption is often hindered by optimization difficulties. First-order methods like Adam [42] typically perform poorly, plateauing at low accuracy [46, 60, 67, 68]. This limitation is generally attributed not to the expressiveness of neural networks, but to the ill-conditioning of the loss [21, 46, 60]. Consequently, specialized optimization strategies are increasingly being advocated [19, 55, 62].

More principled second-order optimization algorithms have only recently begun to emerge [20, 21, 33, 38, 54, 55, 60, 73]. Among them, (Energy) Natural Gradient Descent (NGD) [54] has shown remarkable performance, achieving high accuracy in few iterations by using a search direction of

the form $\mathbf{d} = -(\mathbf{G}(\boldsymbol{\theta}) + \mu\mathbf{I})^{-1}\nabla L(\boldsymbol{\theta})$, where $\boldsymbol{\theta} \in \mathbb{R}^p$ denotes the neural network parameters, L is the loss function, $\mathbf{G}(\boldsymbol{\theta})$ the positive semidefinite Gramian matrix (see Section 2.2), and $\mu > 0$ a regularization parameter. However, solving linear systems with $(\mathbf{G}(\boldsymbol{\theta}) + \mu\mathbf{I})$ using direct methods has a cubic cost $\mathcal{O}(p^3)$, making it impractical for large-scale networks. To mitigate this, matrix-free strategies have been proposed [38, 73], where the system is solved approximately using the conjugate gradient (CG) method using only matrix-vector products (matvecs) with $\mathbf{G}(\boldsymbol{\theta})$, without assembling the matrix. In practice, however, $\mathbf{G}(\boldsymbol{\theta})$ is often ill-conditioned, as shown in Figure 1, leading to slow CG convergence and motivating the need for preconditioning. In this work, we address this issue using tools from Randomized Numerical Linear Algebra (RandNLA) [51, 53]. Our main contributions can be summarized as follows:

- We demonstrate how to efficiently compute matrix-vector products with the Gramian in a more general setting than those considered in [38, 55, 73].
- We propose the use of randomized Nyström preconditioning [26] to construct a preconditioner for $(\mathbf{G}(\boldsymbol{\theta}) + \mu\mathbf{I})$ via a low-rank approximation of the Gramian. This turns the strong spectral decay of the Gramian from a challenge into an asset for our approach.
- We introduce a novel algorithm, *NyströmNGD*, that combines matrix-free NGD with Randomized Nyström preconditioning.
- We present results showing significant improvements over prior NGD-based methods.

Related works Various strategies have been proposed to address training pathologies in PINNs and other neural network-based PDE solvers. These include adaptive quadrature point selection and loss term weighting [13, 66, 67, 68, 69, 71]. More recently, second-order optimizers derived from preconditioning the underlying infinite-dimensional PDE operator have shown promise [21, 33, 54, 55]. Natural Gradient Descent (NGD) has emerged as a unifying framework for these methods, as each can be seen as NGD under a different choice of the optimization metric [55, 56]. Matrix-free implementations have been explored for specific metrics [38, 73], but none of these works addresses the ill-conditioning of the Gramian, which hampers CG convergence in practice. To address this bottleneck, our work bridges NGD with Randomized Numerical Linear Algebra (RandNLA). The use of RandNLA in Machine Learning is well-established [23, 27, 65]. Randomized Nyström preconditioning [26] has been applied to PINNs in [60], but only as a final stage after Adam and L-BFGS, and applying the Nyström approximation, designed for symmetric positive semidefinite matrices, to the Hessian, which may be indefinite. Alternative notions of preconditioning, where the preconditioner is embedded into the loss function, are explored in [7, 47]. K-FAC has also been used to approximate the Gramian [20], but not as a preconditioner, and it requires problem-specific adaptations.

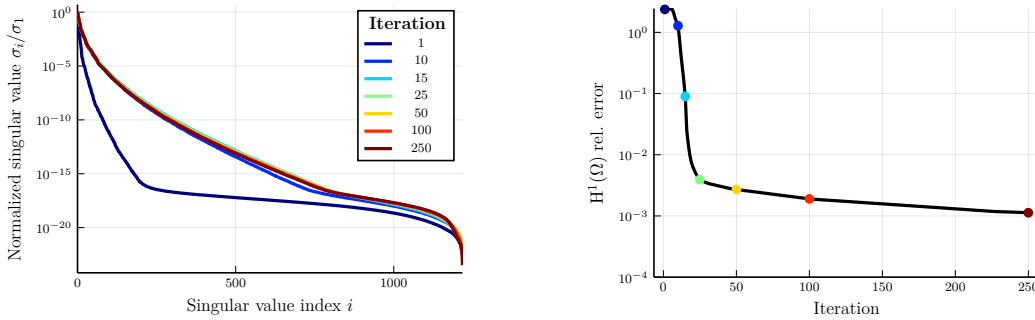


Figure 1: Training of a PINN for the 3D Poisson problem with exact solution $u(x, y, z) = \sin(\pi x) \sin(\pi y) \sin(\pi z)$, using Energy Natural Gradient Descent [54]. Left: spectral decay of the Gramian $\mathbf{G}(\boldsymbol{\theta})$ at different iterations. Right: $H^1(\Omega)$ relative error during training.

2 Preliminaries

2.1 Notation

Let V be a real Hilbert space. Its inner product and norm are denoted by $(\cdot, \cdot)_V$ and $\|\cdot\|_V$. Its (topological) dual is indicated V' , and the duality pairing by $\langle \cdot, \cdot \rangle_{V' \times V}$. The space of bounded linear operators from V to another Hilbert space W is denoted by $\mathcal{L}(V, W)$. We say that $\mathcal{A} \in \mathcal{L}(V, V')$ is symmetric positive semidefinite (SPSD) if the associated bilinear form $a(u, v) = \langle \mathcal{A}u, v \rangle_{V' \times V}$ is symmetric and satisfies $a(u, u) \geq 0$ for all $u \in V$. It is symmetric positive definite (SPD) if a is also coercive, i.e., if $\exists \alpha > 0$ such that $a(u, u) \geq \alpha \|u\|_V^2$ for all $u \in V$.

Let $(\mathcal{M}, g)$ be a smooth Riemannian manifold. The tangent space at $u \in \mathcal{M}$ is denoted by $T_u \mathcal{M}$, with inner product and norm given by $g_u(\cdot, \cdot)$ and $\|\cdot\|_{g_u}$, respectively. For a smooth cost function $E : \mathcal{M} \rightarrow \mathbb{R}$, the Riemannian gradient and Hessian are denoted by $\text{grad}^g E$ and $\text{Hess}^g E$, respectively. The Euclidean gradient of a function $E : \mathbb{R}^n \rightarrow \mathbb{R}$ is denoted by ∇E .

2.2 Natural Gradient Descent

Consider the minimization problem on a Riemannian manifold $(\mathcal{M}, g)$

$$\inf_{u \in \mathcal{M}} E(u). \quad (1)$$

To approximate its solution, we introduce a parametrized ansatz $\mathcal{N} = \{u_\theta := \rho(\theta) : \theta \in \Theta\} \subseteq \mathcal{M}$, where $\rho : \Theta \rightarrow \mathcal{M}$ is a smooth parametrization map. We then consider the optimization problem

$$\inf_{\theta \in \Theta} L(\theta) := \inf_{\theta \in \Theta} E(u_\theta) = \inf_{\theta \in \Theta} (E \circ \rho)(\theta).$$

For instance, u_θ could be a neural network with parameters θ , though the framework applies more generally. We will refer to $\Theta \subseteq \mathbb{R}^p$ as the parameter space, and to $\mathcal{M}$ as the function space.

Natural Gradient Descent (NGD) [3, 50, 55, 56] follows the steepest descent direction according to the metric on $\mathcal{M}$, the “natural” one for the function u_θ , and pulls it back to the parameter space. Let θ be the current iterate. The steepest descent direction in function space would be $-\text{grad}^g E(u_\theta)$. In NGD, the corresponding parameter update direction is $-\mathbf{d}$, where the *natural gradient* $\mathbf{d} \in \mathbb{R}^p$ is such that the first-order expansion $u_{\theta-\alpha \mathbf{d}} \approx u_\theta - \alpha \text{D}\rho(\theta)[\mathbf{d}]$ best approximates (in the metric g) the update in function space. This corresponds to $\text{D}\rho(\theta)[\mathbf{d}] = \sum_{j=1}^p d_j \cdot \partial_{\theta_j} u_\theta$ being the g_{u_θ} -orthogonal projection of $\text{grad}^g E(u_\theta)$ onto the model’s *generalized tangent space* $T_\theta \mathcal{N} = \text{Span}(\{\partial_{\theta_1} u_\theta, \dots, \partial_{\theta_p} u_\theta\}) \subset T_{u_\theta} \mathcal{M}$. As shown, e.g. in [3, 56], the natural gradient is given by $\mathbf{d} = \mathbf{G}(\theta)^\dagger \nabla_\theta L(\theta)$, where $\mathbf{G}(\theta)^\dagger$ is the Moore–Penrose pseudoinverse of the matrix

$$[\mathbf{G}(\theta)]_{ij} = g_{u_\theta}(\partial_{\theta_i} u_\theta, \partial_{\theta_j} u_\theta), \quad (2)$$

often referred to as the Gramian [55, 72] or Fisher information matrix [3, 50]. As observed in the literature [12, 72], it is often beneficial to add a damping term to improve numerical stability and convergence, yielding the regularized direction

$$\mathbf{d} = (\mathbf{G}(\theta) + \mu \mathbf{I})^{-1} \nabla_\theta L(\theta), \quad (3)$$

where $\mu > 0$ is a regularization parameter, typically adapted during the optimization process.

2.3 Randomized Nyström preconditioning

The Gramian matrix $\mathbf{G} = \mathbf{G}(\theta)$ (2) is positive semidefinite and often numerically rank-deficient, as shown in Figure 1. In this setting, Randomized Nyström preconditioning [26] enables efficient construction of a preconditioner for $(\mathbf{G} + \mu \mathbf{I})$ using only matrix-vector products with $\mathbf{G}$. This preconditioner will be used with conjugate gradient to compute the regularized natural gradient (3).

Randomized Nyström approximation Let $\mathbf{G} \in \mathbb{R}^{p \times p}$ be symmetric positive semidefinite (PSD). The *randomized* Nyström approximation [51, Section 14] with a random test matrix $\Omega \in \mathbb{R}^{p \times \ell}$ is

$$\hat{\mathbf{G}}_{\text{nys}} := (\mathbf{G}\Omega) \left(\Omega^\top \mathbf{G}\Omega \right)^\dagger (\mathbf{G}\Omega)^\top. \quad (4)$$

The randomness in Ω ensures that $\hat{\mathbf{G}}_{\text{nys}}$ approximates $\mathbf{G}$ well with high probability [51, Section 14.4]. In this work, we take Ω to be standard normal. This choice, over sampling-based methods [2, 5] and other alternatives [4, 31, 51, 53], is motivated by stronger theoretical guarantees [26, 51] and by the fact that in our setting $\mathbf{G} = \mathbf{G}(\theta)$ in (11) is accessible only via matrix-vector products, and therefore structured test matrices requiring entrywise access are not applicable or less advantageous.

We emphasize that formula (4) is numerically unstable and should not be used directly. A more stable and efficient implementation, which also returns the eigendecomposition of $\hat{\mathbf{G}}_{\text{nys}}$, is described in [51, Algorithm 16] and included in Appendix A.

Randomized Nyström preconditioner Given an eigendecomposition $\hat{\mathbf{G}}_{\text{nys}} = \mathbf{U}\hat{\mathbf{\Lambda}}\mathbf{U}^\top$ with $\hat{\mathbf{\Lambda}} = \text{diag}(\hat{\lambda}_1, \dots, \hat{\lambda}_\ell)$ and $\hat{\lambda}_1 \geq \dots \geq \hat{\lambda}_\ell$, the randomized Nyström preconditioner [26] for $(\mathbf{G} + \mu\mathbf{I})$ with $\mu \geq 0$ is

$$\mathbf{P}^{-1} = (\hat{\lambda}_\ell + \mu)\mathbf{U}(\hat{\mathbf{\Lambda}} + \mu\mathbf{I})^{-1}\mathbf{U}^\top + (\mathbf{I} - \mathbf{U}\mathbf{U}^\top). \quad (5)$$

To ensure good performance in expectation, the sampling parameter ℓ should be proportional to the effective dimension $d_{\text{eff}}(\mu) = \sum_{i=1}^p \lambda_i(\mathbf{G})/(\lambda_i(\mathbf{G}) + \mu)$, which acts as a smoothed count of eigenvalues of $\mathbf{G}$ larger than μ [26, Theorem 1.1]. When $\mathbf{G}$ exhibits strong spectral decay, $d_{\text{eff}}(\mu) \ll p$, making the randomized Nyström preconditioner particularly effective.

Computational complexity Each application of the randomized Nyström preconditioner costs $\mathcal{O}(p\ell)$ flops. Beforehand, one must compute the Nyström approximation. Denoting by C_{mv} the cost of a matrix-vector product with $\mathbf{G}$, the total cost is $\mathcal{O}(C_{\text{mv}}\ell + p\ell^2)$ flops, with memory requirement $\mathcal{O}(p\ell)$. The term $C_{\text{mv}}\ell$ comes from computing $\mathbf{G}\Omega$, i.e., ℓ independent matvecs with $\mathbf{G}$, and is thus embarrassingly parallel. This allows for significant acceleration on multi-core CPUs or GPUs.

3 Matrix-free Natural Gradient Descent

Prior work has shown that matrix-vector products with the Gramian $\mathbf{G}(\theta)$ (2) can be computed via automatic differentiation for specific metrics [38, 55, 73]. We present a unified framework for metric selection and show that autodiff-based matvecs extend to broader metric classes.

3.1 Choice of the Metric

In some cases, the problem’s structure naturally suggests a metric, as in Variational MonteCarlo [36, 55, 58] or Sobolev gradient flows [17, 35]. In general, the link between NGD and Riemannian gradient descent [54] suggests selecting the metric g to reduce the condition number of the Riemannian Hessian [14]. Further details are deferred to Appendix B.

Linear problems Consider a linear problem $\mathcal{A}u = f \in W'$, where V, W are Hilbert spaces, and $\mathcal{A} : V \rightarrow W'$ is a linear isomorphism. The associated least-squares problem and natural metric are

$$\inf_{u \in V} E(u) := \frac{1}{2} \|\mathcal{A}u - f\|_{W'}^2, \quad g(v, w) := (\mathcal{A}v, \mathcal{A}w)_{W'}. \quad (6)$$

If $V = W$ and $\mathcal{A}$ is symmetric positive definite, one can endow V with the $\mathcal{A}$ -induced inner product $(v, w)_{\mathcal{A}} = \langle \mathcal{A}v, w \rangle_{V' \times V}$, and use it as the NGD metric. Then, up to an additive constant irrelevant for optimization, the loss (6) coincides with the variational form used in the Deep Ritz method [25]

$$\inf_{u \in V} E(u) := \frac{1}{2} \langle \mathcal{A}u, u \rangle_{V' \times V} - \langle f, u \rangle_{V' \times V}. \quad (7)$$

A link with operator preconditioning NGD can be viewed as a preconditioned gradient descent, where the preconditioner is the Gramian [21]. For a *linear* parametrized model $u_\theta = \sum_{i=1}^p \theta_i \varphi_i$, the Gramian becomes $[\mathbf{G}(\theta)]_{ij} = g_{u_\theta}(\varphi_i, \varphi_j)$. If $(\mathcal{M}, g) = (V, (\cdot, \cdot)_V)$ and g is induced by a positive definite linear operator $\mathcal{G} : V \rightarrow V'$, then $\mathbf{G}(\theta) = \mathbf{G}$ where $[\mathbf{G}]_{ij} = \langle \mathcal{G}\varphi_i, \varphi_j \rangle_{V' \times V}$ does not depend on θ . This coincides with preconditioners arising from an operator preconditioning perspective, in the context of Galerkin discretizations of PDEs [30, 37, 43, 48].

Nonlinear problems If $\mathcal{M}$ is equipped with a metric a and $\text{Hess}^a E(u)$ is positive definite for all $u \in \mathcal{M}$, Newton’s method [14] is induced by choosing the metric

$$g_u(v, w) = \text{Hess}^a E(u)[v, w]. \quad (8)$$

In the case $(\mathcal{M}, a) = (V, (\cdot, \cdot)_V)$ for a Hilbert space V , this leads to the Energy Natural Gradient Descent (ENGd) method [54, 55]. Note that to interpret ENGd as a Natural Gradient Descent, it is necessary that $\text{Hess}^a E(u)$ is SPD for all $u \in \mathcal{M}$, as otherwise (8) does not define a valid metric.

Consider now the nonlinear problem $\mathcal{R}(u) = 0$, where $\mathcal{R} : V \rightarrow W$ is a (possibly nonlinear) residual between Hilbert spaces V, W . For the least-squares functional $E(u) = \frac{1}{2} \|\mathcal{R}(u)\|_W^2$, a symmetric positive semidefinite approximation of the Hessian is the Gauss-Newton one [1, 70]:

$$g_u(v, w) = (D\mathcal{R}(u)[v], D\mathcal{R}(u)[w])_W. \quad (9)$$

This choice of the metric leads the Gauss-Newton NGD method [33, 38]. Note that g_u is, in general, only semidefinite. Positive definiteness of g_u is necessary to interpret it as a Riemannian metric, and whether this condition holds depends on the structure of the underlying nonlinear PDE.

3.2 Matrix-free formulation

We show that matrix-vector products with the Gramian (2) can be computed via automatic differentiation, provided the metric satisfies the following assumption.

Assumption 3.1. Let $\Omega \subset \mathbb{R}^d$ be a domain, and let $\mathcal{M} \subseteq V$, where V is a Hilbert space of functions on Ω . Assume that the metric g_u takes the form

$$g_u(v, w) = \int_{\Omega} \mathcal{F}_u v(\mathbf{x}) \cdot \mathcal{F}_u w(\mathbf{x}) \varphi_u(\mathbf{x}) \, d\mathbf{x}, \quad (10)$$

for all $v, w \in T_u \mathcal{M} \subseteq V$, where $\mathcal{F}_u : V \rightarrow L^2(\Omega; \varphi)^k$ is linear and continuous.

Theorem 3.2. Let Assumption 3.1 hold, and let $\text{sg} : \mathbb{R}^p \rightarrow \mathbb{R}^p$ denote the stop-gradient operator, formally defined by $\text{sg}(\boldsymbol{\theta}) = \boldsymbol{\theta}$ and $\nabla \text{sg}(\boldsymbol{\theta}) = \mathbf{0}$. If the integral (10) is approximated using a quadrature rule with points $\{\mathbf{x}_r\}_{r=1}^q$ and weights $\{w_r\}_{r=1}^q$, then

$$\mathbf{G}(\boldsymbol{\theta}) = \mathbf{J}_{\boldsymbol{\theta}} \mathbf{F}(\boldsymbol{\theta})^T \begin{bmatrix} w_1 \varphi_{u_{\boldsymbol{\theta}}}(\mathbf{x}_1) \mathbf{I}_k & & \\ & \ddots & \\ & & w_q \varphi_{u_{\boldsymbol{\theta}}}(\mathbf{x}_q) \mathbf{I}_k \end{bmatrix} \mathbf{J}_{\boldsymbol{\theta}} \mathbf{F}(\boldsymbol{\theta}), \quad \mathbf{F}(\boldsymbol{\theta}) = \begin{bmatrix} \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_{\boldsymbol{\theta}}(\mathbf{x}_1) \\ \vdots \\ \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_{\boldsymbol{\theta}}(\mathbf{x}_q) \end{bmatrix}, \quad (11)$$

where $\mathbf{J}_{\boldsymbol{\theta}}$ denotes the Jacobian with respect to $\boldsymbol{\theta}$.

The proof of Theorem 3.2 is deferred to Appendix C. As a consequence of Theorem 3.2, matrix-vector products with $\mathbf{G}(\boldsymbol{\theta})$ can be computed using only products with the Jacobian and the transpose Jacobian of $\mathbf{F}$. These operations can be efficiently implemented via automatic differentiation, using forward-mode autodiff for Jacobian-vector products (JVPs) and backward-mode autodiff for vector-Jacobian products (VJPs). Note that most autodiff frameworks include a stop-gradient function.

Remark 3.3. Assumption 3.1 encompasses standard metrics used in PDE-related function spaces. For instance, the $L^2(\Omega)$ and $H^1(\Omega)$ metrics correspond to the choices $\mathcal{F}_u v = v$ and $\mathcal{F}_u v = (\nabla v, v)$, respectively. It also includes the Gauss-Newton metric (9) on $L^2(\Omega)$, obtained by taking $\mathcal{F}_u = D\mathcal{R}(u)$, for which matrix-free NGD implementations have been proposed in [38, 55] for specific problems. However, Assumption 3.1 is significantly more general and covers a much broader class of metrics. In practice, it includes many of the metrics relevant in the context of PDEs, since these typically need to be expressible as integrals to be computationally tractable. For example, it includes the Newton metric $g_u(v, w) = \int_{\Omega} \nabla v \cdot \nabla w + 3u^2 v w$ in [55, Section 4.2], and the energy inner product for the Gross-Pitaevskii equation as considered in [35], $g_u(v, w) = \int_{\Omega} \nabla v \cdot \nabla w + V v w + \beta |u|^2 v w$.

Computational Cost With automatic differentiation, the cost of each JVP and VJP is a small constant factor (typically between 2 and 4) times the cost of evaluating $\mathbf{F}_i(\boldsymbol{\theta})$ for $i = 1, 2$ [29, 49]. For a multilayer perceptron, the evaluation cost scales as $\mathcal{O}(pq)$, where p is the number of trainable parameters and q is the number of quadrature points. Therefore, the cost of each matrix-vector product with $\mathbf{G}(\boldsymbol{\theta})$ is $C_{\text{mv}} = \mathcal{O}(pq)$. Note however that when executed on modern GPUs, batch evaluation across q points benefits from parallelization.

4 NyströmNGD: Matrix-Free Natural Gradient Descent with Randomized Nyström Preconditioning

To compute the regularized natural gradient, we approximately solve $(\mathbf{G}(\boldsymbol{\theta}) + \mu\mathbf{I})\mathbf{d} = \nabla L(\boldsymbol{\theta})$ using the preconditioned conjugate gradient (PCG) method in a matrix-free fashion, with the preconditioner constructed via randomized Nyström. This approach leverages the structure of the problem: matrix-vector products with the Gramian $\mathbf{G}(\boldsymbol{\theta})$ can be computed efficiently via automatic differentiation, while the strong spectral decay commonly observed in practice (see Figure 1) permits an accurate low-rank approximation, yielding a preconditioner that is both efficient to apply and effective in accelerating convergence. This section presents the resulting algorithm in full detail.

Choice of the regularization parameter μ The regularization parameter $\mu > 0$ is crucial for numerical stability and for mitigating the slow initial convergence typical of quasi-second-order methods like NGD. Prior works proposed heuristics that adapt μ dynamically [12, 28] or decrease it with the loss [72], so that early iterations mimic gradient descent and gradually shift to NGD near a local minimum. We instead propose a heuristic based on the condition number of $(\mathbf{G}(\boldsymbol{\theta}) + \mu\mathbf{I})$.

Letting $\lambda_1 \geq \dots \geq \lambda_p$ denote the eigenvalues of the Gramian $\mathbf{G}(\boldsymbol{\theta})$, the spectral condition number of $(\mathbf{G}(\boldsymbol{\theta}) + \mu\mathbf{I})$ is $k = (\lambda_1 + \mu)/(\lambda_p + \mu) \leq 1 + \lambda_1/\mu$. Hence, choosing $\mu = c \cdot \lambda_1$ guarantees $k \leq 1 + c^{-1}$. In practice, good results are obtained by setting c proportional to machine epsilon, i.e., $\mu = \gamma \cdot \epsilon_{\text{mach}} \cdot \lambda_1$, with $\gamma = p$ as a reasonable default. Observe that although λ_1 is not directly available, it can be estimated by $\hat{\lambda}_1$ from randomized Nyström. It can be shown that the expected relative error can be small even for modest values of ℓ , when $\mathbf{G}(\boldsymbol{\theta})$ exhibits strong spectral decay [51, Theorem 14.1].

Choice of rank parameter ℓ The rank parameter ℓ controls the computational cost of the preconditioner and should ideally scale with the effective dimension $d_{\text{eff}}(\mu)$ [26, Theorem 1.1]. As $d_{\text{eff}}(\mu)$ is not directly accessible, we estimate ℓ adaptively. Following [26, Section 5.4.2], we do it by monitoring the ratio $\hat{\lambda}_\ell/\mu$: if it drops below 10, we double ℓ for the next iteration, up to a prescribed maximum ℓ_{max} . Otherwise, we reduce ℓ to the smallest index satisfying $\hat{\lambda}_\ell/\mu < 10$, adding a small offset (e.g., 1 or 2) to avoid oscillations.

Other algorithmic details and pseudocode Algorithm 1 summarizes the proposed algorithm. At each iteration, we construct the randomized Nyström preconditioner (lines 4 to 6) using the matrix-free Gramian. We then apply preconditioned conjugate gradient (PCG) to solve the linear system (3) (line 8). The PCG stopping criterion combines a maximum number of iterations `maxit` and a relative tolerance `rel_tol` $= \min(\kappa, \|\nabla L(\boldsymbol{\theta}_k)\|_2)$, inspired by the Steihaug–Toint truncated CG method [18]. This design avoids overspending computation far from critical points, where NGD steps may underperform compared to GD, and ensures accuracy near local minima. Once the search direction $\mathbf{d}_k$ is computed, a line search is performed to update the iterate (lines 9 and 10). In practice, we found that a standard backtracking line-search [70] suffices, without requiring logarithmic grid searches as in [38, 54]. Finally, the rank parameter ℓ is updated adaptively (lines 11 to 15).

5 Numerical Experiments

We test NyströmNGD on a range of forward and inverse PDE problems with known analytical solutions. Additional resources and implementation details are provided in Appendix E. Additional numerical experiments involving variational formulations of PDEs are presented in Appendix F.

Comparison of the methods We compare NyströmNGD against Natural Gradient Descent with SVD-based pseudoinverse (legend: NGD), matrix-free NGD solved via conjugate gradient without preconditioning (legend: NGD with CG), and the BFGS algorithm [70]. We run NGD-based methods for 500 or 1000 iterations, depending on the computational cost, and BFGS for 6×10^4 or 1.2×10^5 iterations. For a fair comparison with NyströmNGD, in NGD with CG we use the same relative tolerance as in Algorithm 1 and set the maximum number of iterations to `maxit` + ℓ_{max} . Given the different per-iteration cost of the methods, we compare them in both iteration count and wall-clock time. We do not include first-order optimizers (e.g., gradient descent, Adam) or L-BFGS, as they have been shown to be non-competitive [38, 55]. In all experiments, we employ fully connected neural

Algorithm 1 NyströmNGD

Require: Initial parameter $\theta_0 \in \mathbb{R}^p$, initial rank ℓ_0 , maximum rank $\ell_{\max}$, $\gamma \in \mathbb{R}_+$ (e.g. $\gamma = p$), CG parameters maxit, κ (e.g. $\text{maxit} = 20, \kappa = 0.1$)

```
1: Initialize  $\ell = \ell_0$ 
2: for  $k = 0, 1, 2, \dots$  do
3:   Define  $\mathbf{G}_{\text{fun}} = \mathbf{v} \mapsto \mathbf{G}(\theta)\mathbf{v}$  via (11) ▷ Matrix-free Gramian
4:    $[\mathbf{U}, \hat{\mathbf{A}}] = \text{RandomizedNyström}(\mathbf{G}_{\text{fun}}, \ell)$  ▷ Randomized Nyström approximation
5:    $\mu = \gamma \cdot \hat{\lambda}_1 \cdot \epsilon_{\text{mach}}$  ▷ Regularization parameter  $\mu$ 
6:    $\mathbf{P}_{\text{fun}} = \mathbf{v} \mapsto \frac{1}{\lambda_\ell + \mu} \mathbf{U}(\hat{\mathbf{A}} + \mu \mathbf{I})\mathbf{U}^\top \mathbf{v} + (\mathbf{I} - \mathbf{U}\mathbf{U}^\top)\mathbf{v}$  ▷ Matrix-free preconditioner
7:    $\text{rel\_tol} = \min(\kappa, \|\nabla L(\theta_k)\|_2)$  ▷ Relative tolerance for PCG
8:    $\mathbf{d}_k = \text{PCG}(\mathbf{G}_{\text{fun}} + \mu \cdot \text{Id}, \nabla L(\theta_k), \text{rel\_tol}, \text{maxit}, \mathbf{P}_{\text{fun}})$  ▷ Run PCG
9:    $\alpha_k = \text{Linesearch}(\theta_k, -\mathbf{d}_k)$  ▷ Linesearch along  $\alpha \mapsto \theta_k - \alpha \mathbf{d}_k$ 
10:   $\theta_{k+1} = \theta_k - \alpha_k \mathbf{d}_k$ 
11:  if  $\hat{\lambda}_\ell < 10\mu$  then
12:     $\ell = \min(2\ell, \ell_{\max})$  ▷ Possibly increase rank parameter  $\ell$ 
13:  else
14:     $\ell = \min(\min\{i : \hat{\lambda}_i > 10\mu\} + 1, \ell_{\max})$  ▷ Possibly decrease rank parameter  $\ell$ 
15:  end if
16: end for
```

networks with tanh activation and two hidden layers of width 32. Integrals in both the loss and the metric are discretized via Monte Carlo quadrature. All experiments are averaged over 10 runs. In the plots, the solid line denotes the median, and the shaded region spans the first and third quantiles.

Computational details For the PINN formulations, we use JAX [15] and JAXopt [11]. The JAXopt BFGS implementation was optimized by us for improved efficiency, as detailed in Appendix D, yielding, together with JIT compilation, a $1000\times$ speedup over the timings reported in [54]. The NGD implementation is taken from the public repository of [72]. All experiments were conducted in double precision on a single NVIDIA RTX 4090 GPU with 24GB of memory.

5.1 Poisson problem in 3D

We consider the Poisson problem on the 3D unit cube

$$\begin{cases} -\Delta u = f & \Omega = (0, 1)^3, \\ u = g_D & \partial\Omega, \end{cases} \quad (12)$$

with exact solution $u(x, y, z) = \sin(\pi x) \sin(\pi y) \sin(\pi z)$ as in [72]. The PINNs least-squares loss and the corresponding metric are

$$E(u) = \frac{1}{2} \|\Delta u + f\|_{L^2(\Omega)}^2 + \frac{1}{2} \|u - g_D\|_{L^2(\partial\Omega)}^2,$$
$$g(u, v) = (\Delta u, \Delta v)_{L^2(\Omega)} + (u, v)_{L^2(\partial\Omega)} = \frac{1}{2} \int_{\Omega} \Delta u \Delta v \, d\mathbf{x} + \frac{1}{2} \int_{\partial\Omega} u v \, d\mathbf{s}.$$

Further details are provided in Appendix E.1.

As shown in Figure 2, NyströmNGD yields the best results in both final accuracy and computational efficiency, converging in around 100 iterations. The Nyström rank parameter ℓ increases rapidly during early iterations, consistent with Figure 1, and then stabilizes well below the maximum allowed rank $\ell_{\max} = 500$. This confirms the effectiveness of the adaptive rank selection. NGD initially converges faster in iterations but plateaus early, while NGD with CG is slower due to the many CG steps needed to approximate the Gramian inverse. BFGS is not competitive in iteration count, though its lower per-iteration cost allows for faster initial progress in wall-clock time. However, it ultimately stagnates at a higher error.

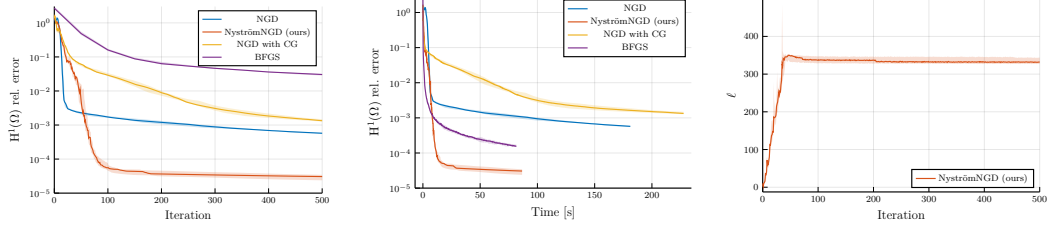


Figure 2: Poisson equation in 3D discretized via PINNs. From left to right: relative $H^1(\Omega)$ error vs. iterations (first 500 iterations), relative $H^1(\Omega)$ error vs. wall-clock time, and Nyström rank parameter ℓ vs. iterations, with $\ell_{\max} = 500$.

5.2 Heat equation in 3+1D

We consider the heat equation

$$\begin{cases} \partial_t u - \Delta u = f & I \times \Omega = (0, 1) \times (0, 1)^3, \\ u = g_D & I \times \partial\Omega, \\ u(0, \cdot) = u_0 & \Omega, \end{cases} \quad (13)$$

with exact solution $u(t, x, y, z) = (\cos(\pi x) + \cos(\pi y) + \cos(\pi z)) e^{-\pi^2 t/4}$ as in [72]. The corresponding least squares loss and metrics are

$$\begin{aligned} E(u) &= \frac{1}{2} \|\partial_t u - \Delta u - f\|_{L^2(I, L^2(\Omega))}^2 + \frac{1}{2} \|u - g_D\|_{L^2(I, L^2(\partial\Omega))}^2 + \frac{1}{2} \|u(0, \cdot) - u_0\|_{L^2(\Omega)}^2, \\ g(u, v) &= (\partial_t u - \Delta u, \partial_t v - \Delta v)_{L^2(I, L^2(\Omega))} + (u, v)_{L^2(I, L^2(\Omega))} + (u(0, \cdot), v(0, \cdot))_{L^2(\Omega)}. \end{aligned}$$

Further details are provided in Appendix E.2.

Also in this example, NyströmNGD provides the best accuracy at a reduced computational cost, as shown in Figure 3. All other methods saturate at a lower accuracy, despite being given a higher computational budget. The Nyström rank parameter ℓ rapidly increases and then stabilizes at a value lower than $\ell_{\max} = 500$, although at a higher value compared to the Poisson example.

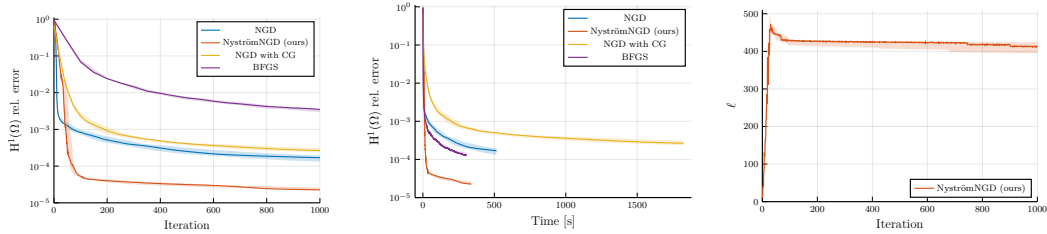


Figure 3: Heat equation in 3+1D discretized via PINNs. From left to right: relative $H^1(\Omega)$ error vs. iterations (first 1000 iterations), relative $H^1(\Omega)$ error vs. wall-clock time, and Nyström rank parameter ℓ vs. iterations, with $\ell_{\max} = 500$.

5.2.1 Kovasznay Flow (2D steady state Navier–Stokes flow)

We consider the two-dimensional steady Navier–Stokes flow as originally described in [45], with Reynolds number $\text{Re} = 40$, following the setup in [38, Section 4.1]. The system of PDEs reads

$$\begin{cases} -\nu \Delta u + (u \cdot \nabla)u + \nabla p = f & \Omega = (-\frac{1}{2}, 1) \times (-\frac{1}{2}, \frac{3}{2}), \\ \nabla \cdot u = 0 & \Omega, \\ u = g_D & \partial\Omega. \end{cases} \quad (14)$$

The nonlinear least squares loss is

$$E(u, p) = \frac{1}{2} \|-\nu \Delta u + (u \cdot \nabla)u + \nabla p - f\|_{L^2(\Omega)^2}^2 + \frac{1}{2} \|\nabla \cdot u\|_{L^2(\Omega)}^2 + \frac{1}{2} \|u - g_D\|_{L^2(\partial\Omega)}^2,$$

and as a metric we consider the Gauss–Newton metric (9). We employ two separate neural networks for u and p . Further details are provided in Appendix E.3.

NyströmNGD still provides the best accuracy in fewer than 1000 iterations, as shown in Figure 4. However, compared to previous examples, the rank ℓ grows to the maximum rank $\ell_{\max} = 500$ before stabilizing around 400, indicating that the Gramian is less suited to low-rank approximation than in previous examples. This explains the smaller difference, both in final accuracy and computation time, between NyströmNGD and BFGS. Nevertheless, the gap in iterations remains substantial.

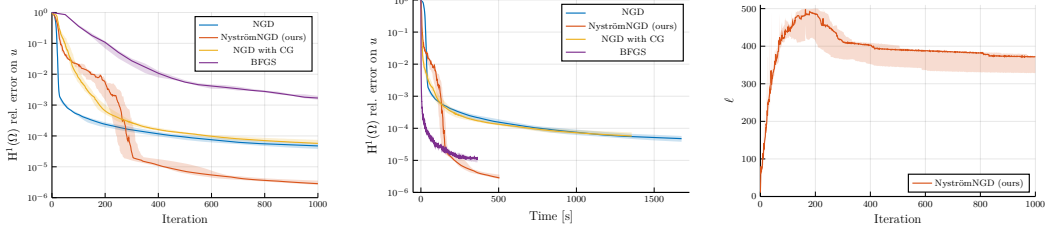


Figure 4: Kovasnay Flow discretized via PINNs. From left to right: relative $H^1(\Omega)$ error on u vs. iterations (first 1000 iterations), relative $H^1(\Omega)$ error on u vs. wall-clock time, and Nyström rank parameter ℓ vs. iterations, with $\ell_{\max} = 500$.

5.3 Source recovery problem for the Poisson equation in 3D

We consider the inverse problem of recovering both the solution u and the source term f in the 3D Poisson equation (12) from $N_{\text{data}} = 40$ noiseless observations $\{u_i^*\}_{i=1}^{N_{\text{data}}}$ of the true solution at random points $\{\mathbf{x}_i\}_{i=1}^{N_{\text{data}}}$ in Ω . The corresponding least squares loss and metrics are

$$E(u, f) = \frac{1}{2} \|\Delta u + f\|_{L^2(\Omega)}^2 + \frac{1}{2} \|u - g_D\|_{L^2(\partial\Omega)}^2 + \frac{1}{2N_{\text{data}}} \sum_{i=1}^{N_{\text{data}}} |u(\mathbf{x}_i) - u_i^*|^2$$

$$g((u, f), (v, h)) = (\Delta u + f, \Delta v + h)_{L^2(\Omega)} + (u, v)_{L^2(\partial\Omega)} + (u, v)_{L^2(\Omega)}.$$

We employ two separate neural networks for u and f . Further details are provided in Appendix E.4.

As shown in Figure 5, NyströmNGD achieves higher accuracy than NGD and NGD with CG, and converges in fewer iterations than BFGS. However, BFGS can perform more iterations within the same time budget and eventually surpasses NyströmNGD in accuracy. The reduced performance of NyströmNGD can be explained by the poor low-rank approximability of the Gramian, as indicated by the rank parameter ℓ reaching the threshold $\ell_{\max} = 500$. This suggests that $\ell_{\max}$ matrix-vector products are insufficient for accurately approximating the Gramian via randomized Nyström, leading to a less effective preconditioner.

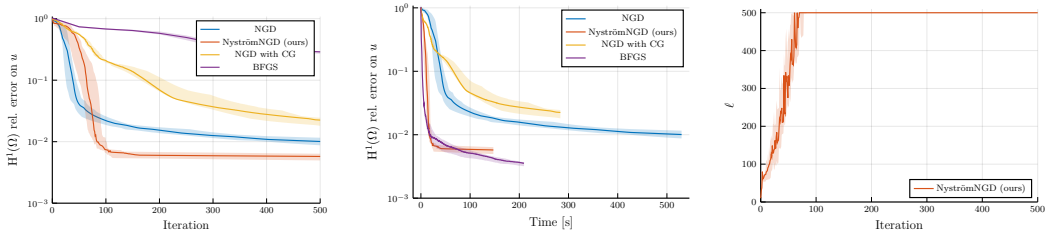


Figure 5: Source recovery problem for the Poisson equation in 3D discretized via PINNs. From left to right: relative $H^1(\Omega)$ error on u vs. iterations (first 500 iterations), relative $H^1(\Omega)$ error on u vs. wall-clock time, and Nyström rank parameter ℓ vs. iterations, with $\ell_{\max} = 500$.

6 Discussion and Conclusion

We introduced *NyströmNGD*, a novel optimization algorithm that uses randomized Nyström preconditioning to accelerate the inner conjugate gradient (CG) solver in matrix-free natural gradient

descent (NGD). This approach significantly reduces the cost of the inner solver, resulting in faster convergence in both iterations and wall-clock time. Moreover, our work extends matrix-free NGD to a broader class of metrics relevant to PDE-related problems. We validated our method on different PDE problems discretized using PINNs, demonstrating that NyströmNGD consistently outperforms prior NGD methods by achieving higher accuracy at a lower computational cost.

Limitations and future work The use of randomized Nyström for preconditioning builds on the empirically observed low-rank structure of the Gramian. Our results confirm that in many cases the Gramian admits strong low-rank compression, though this is not always guaranteed and warrants further analysis. Nevertheless, the core idea of preconditioning the inner CG solver remains applicable, and we plan to explore alternative Gramian approximations for use as preconditioners. Notably, NyströmNGD not only reduces the computational cost but also improves final accuracy over standard NGD. This may be due to improved numerical stability or the stochastic nature of the Nyström approximation, which could help escape poor local minima, similar to how SGD can outperform full-batch GD in traditional machine learning [34, 39, 44]. This aspect merits further investigation. Finally, NyströmNGD is especially promising in the context of neural network-based solvers for variational formulations of PDEs, where the choice of metric can be guided by the underlying variational structure. Our preliminary results (see Appendix F) support the potential of the proposed approach and motivate future research in this direction.

Acknowledgments and Disclosure of Funding

This work has received funding from the European Union’s Horizon Europe research and innovation programme under the Marie Skłodowska-Curie Action MSCA-DN-101119556 (IN-DEEP). The authors are members of the Gruppo Nazionale Calcolo Scientifico - Istituto Nazionale di Alta Matematica (GNCS-INdAM). C. Marcati and G. Sangalli acknowledge support from the Italian MUR through the PRIN 2022 PNRR project NOTES (No. P2022NC97R). G. Sangalli further acknowledges support from the Italian MUR through PNRR-M4C2-I1.4-NC-HPC-Spoke6. C. Marcati further acknowledges the PRIN 2022 project ASTICE (No. 202292JW3F).

References

- [1] P.-A. Absil, R. Mahony, and R. Sepulchre. *Optimization Algorithms on Matrix Manifolds*. Princeton University Press, December 2008. ISBN 978-1-4008-3024-4. doi: 10.1515/9781400830244.
- [2] Ahmed El Alaoui and Michael W. Mahoney. Fast Randomized Kernel Methods With Statistical Guarantees, November 2015.
- [3] Shun-ichi Amari. Natural Gradient Works Efficiently in Learning. *Neural Computation*, 10(2): 251–276, February 1998. ISSN 0899-7667. doi: 10.1162/089976698300017746.
- [4] Haim Avron, Kenneth L. Clarkson, and David P. Woodruff. Faster Kernel Ridge Regression Using Sketching and Preconditioning, July 2017.
- [5] Francis Bach. Sharp analysis of low-rank kernel matrix approximations, May 2013.
- [6] Santiago Badia, Wei Li, and Alberto F. Martín. Adaptive Finite Element Interpolated Neural Networks, March 2024.
- [7] Santiago Badia, Wei Li, and Alberto F. Martín. Finite element interpolated neural networks for solving forward and inverse problems. *Computer Methods in Applied Mechanics and Engineering*, 418:116505, January 2024. ISSN 0045-7825. doi: 10.1016/j.cma.2023.116505.
- [8] Jules Berman, Paul Schwerdtner, and Benjamin Peherstorfer. Neural Galerkin schemes for sequential-in-time solving of partial differential equations with deep networks. In *Handbook of Numerical Analysis*, volume 25, pages 389–418. Elsevier, 2024. ISBN 978-0-443-23984-7. doi: 10.1016/bs.hna.2024.05.006.

- [9] S. Berrone, C. Canuto, M. Pintore, and N. Sukumar. Enforcing Dirichlet boundary conditions in physics-informed neural networks and variational physics-informed neural networks. *Heliyon*, 9(8):e18820, August 2023. ISSN 2405-8440. doi: 10.1016/j.heliyon.2023.e18820.
- [10] Stefano Berrone, Claudio Canuto, and Moreno Pintore. Variational Physics Informed Neural Networks: the Role of Quadratures and Test Functions. *Journal of Scientific Computing*, 92(3): 100, August 2022. ISSN 1573-7691. doi: 10.1007/s10915-022-01950-4.
- [11] Mathieu Blondel, Quentin Berthet, Marco Cuturi, Roy Frostig, Stephan Hoyer, Felipe Llinares-López, Fabian Pedregosa, and Jean-Philippe Vert. Efficient and Modular Implicit Differentiation. *arXiv preprint arXiv:2105.15183*, 2021.
- [12] Andrea Bonfanti, Giuseppe Bruno, and Cristina Cipriani. The Challenges of the Nonlinear Regime for Physics-Informed Neural Networks, October 2024.
- [13] Andrea Bonito, Ronald DeVore, Guergana Petrova, and Jonathan W. Siegel. Convergence and error control of consistent PINNs for elliptic PDEs, January 2025.
- [14] Nicolas Boumal. *An Introduction to Optimization on Smooth Manifolds*. Cambridge University Press, 1 edition, March 2023. ISBN 978-1-00-916616-4. doi: 10.1017/9781009166164.
- [15] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- [16] Ziang Chen, Jianfeng Lu, and Yulong Lu. On the representation of solutions to elliptic pdes in barron spaces. In *Proceedings of the 35th International Conference on Neural Information Processing Systems*, NeurIPS ’21, Red Hook, NY, USA, 2021. Curran Associates Inc. ISBN 9781713845393.
- [17] Ziang Chen, Jianfeng Lu, Yulong Lu, and Xiangxiong Zhang. On the Convergence of Sobolev Gradient Flow for the Gross–Pitaevskii Eigenvalue Problem. *SIAM Journal on Numerical Analysis*, 62(2):667–691, April 2024. ISSN 0036-1429, 1095-7170. doi: 10.1137/23M1552553.
- [18] Andrew R. Conn, Nicholas I. M. Gould, and Philippe L. Toint. *Trust Region Methods*. Society for Industrial and Applied Mathematics, January 2000. ISBN 978-0-89871-460-9. doi: 10.1137/1.9780898719857.
- [19] Salvatore Cuomo, Vincenzo Schiano Di Cola, Fabio Giampaolo, Gianluigi Rozza, Maziar Raissi, and Francesco Piccialli. Scientific Machine Learning Through Physics-Informed Neural Networks: Where we are and What’s Next. *Journal of Scientific Computing*, 92(3):88, July 2022. ISSN 1573-7691. doi: 10.1007/s10915-022-01939-z.
- [20] Felix Dangel, Johannes Müller, and Marius Zeinhofer. Kronecker-Factored Approximate Curvature for Physics-Informed Neural Networks, October 2024.
- [21] Tim De Ryck, Florent Bonnet, Siddhartha Mishra, and Emmanuel de Bézenac. An operator preconditioning perspective on training in physics-informed machine learning, May 2024.
- [22] Tim De Ryck, Siddhartha Mishra, and Roberto Molinaro. wPINNs: Weak Physics Informed Neural Networks for Approximating Entropy Solutions of Hyperbolic Conservation Laws. *SIAM Journal on Numerical Analysis*, 62(2):811–841, April 2024. ISSN 0036-1429. doi: 10.1137/22M1522504.
- [23] Michał Dereziński and Michael W. Mahoney. Recent and Upcoming Developments in Randomized Numerical Linear Algebra for Machine Learning, June 2024.
- [24] M. W. M. G. Dissanayake and N. Phan-Thien. Neural-network-based approximations for solving partial differential equations. *Communications in Numerical Methods in Engineering*, 10(3):195–201, 1994. ISSN 1099-0887. doi: 10.1002/cnm.1640100303.
- [25] Weinan E and Bing Yu. The Deep Ritz method: A deep learning-based numerical algorithm for solving variational problems, September 2017.

- [26] Zachary Frangella, Joel A. Tropp, and Madeleine Udell. Randomized Nyström Preconditioning. *SIAM Journal on Matrix Analysis and Applications*, 44(2):718–752, June 2023. ISSN 0895-4798. doi: 10.1137/21M1466244.
- [27] Zachary Frangella, Pratik Rathore, Shipu Zhao, and Madeleine Udell. SketchySGD: Reliable Stochastic Optimization via Randomized Curvature Estimates, February 2024.
- [28] Henri P Gavin. The Levenberg-Marquardt algorithm for nonlinear least squares curve-fitting problems. *Department of Civil and Environmental Engineering Duke University August*, 3: 1–23, 2019.
- [29] Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation, Second Edition*. Society for Industrial and Applied Mathematics, second edition, January 2008. ISBN 978-0-89871-659-7 978-0-89871-776-1. doi: 10.1137/1.9780898717761.
- [30] Andreas Günnel, Roland Herzog, and Ekkehard Sachs. A note on preconditioners and scalar products in Krylov subspace methods for self-adjoint problems in Hilbert space. *Electron. Trans. Numer. Anal.*, 41:13–20, 2014.
- [31] N. Halko, P. G. Martinsson, and J. A. Tropp. Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions. *SIAM Review*, 53(2):217–288, January 2011. ISSN 0036-1445. doi: 10.1137/090771806.
- [32] Jiequn Han, Arnulf Jentzen, and Weinan E. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34): 8505–8510, August 2018. doi: 10.1073/pnas.1718942115.
- [33] Wenrui Hao, Qingguo Hong, and Xianlin Jin. Gauss Newton Method for Solving Variational Problems of PDEs with Neural Network Discretizations. *Journal of Scientific Computing*, 100 (1):17, June 2024. ISSN 1573-7691. doi: 10.1007/s10915-024-02535-z.
- [34] Moritz Hardt, Benjamin Recht, and Yoram Singer. Train faster, generalize better: Stability of stochastic gradient descent, February 2016.
- [35] Patrick Henning and Daniel Peterseim. Sobolev Gradient Flow for the Gross–Pitaevskii Eigenvalue Problem: Global Convergence and Computational Efficiency. *SIAM Journal on Numerical Analysis*, 58(3):1744–1772, January 2020. ISSN 0036-1429, 1095-7170. doi: 10.1137/18M1230463.
- [36] Jan Hermann, Zeno Schätzle, and Frank Noé. Deep-neural-network solution of the electronic Schrödinger equation. *Nature Chemistry*, 12(10):891–897, October 2020. ISSN 1755-4349. doi: 10.1038/s41557-020-0544-y.
- [37] R. Hiptmair. Operator Preconditioning. *Computers & Mathematics with Applications*, 52(5): 699–706, September 2006. ISSN 0898-1221. doi: 10.1016/j.camwa.2006.10.008.
- [38] Anas Jnini, Flavio Vella, and Marius Zeinhofer. Gauss-Newton Natural Gradient Descent for Physics-Informed Computational Fluid Dynamics, February 2024.
- [39] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima, February 2017.
- [40] E. Kharazmi, Z. Zhang, and G. E. Karniadakis. Variational Physics-Informed Neural Networks For Solving Partial Differential Equations, November 2019.
- [41] Ehsan Kharazmi, Zhongqiang Zhang, and George E. M. Karniadakis. hp-VPINNs: Variational physics-informed neural networks with domain decomposition. *Computer Methods in Applied Mechanics and Engineering*, 374:113547, February 2021. ISSN 0045-7825. doi: 10.1016/j.cma.2020.113547.
- [42] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization, January 2017.

- [43] Robert C. Kirby. From Functional Analysis to Iterative Methods. *SIAM Review*, 52(2):269–293, January 2010. ISSN 0036-1445. doi: 10.1137/070706914.
- [44] Bobby Kleinberg, Yuanzhi Li, and Yang Yuan. An Alternative View: When Does SGD Escape Local Minima? In *Proceedings of the 35th International Conference on Machine Learning*, pages 2698–2707. PMLR, July 2018.
- [45] L. I. G. Kovasznay. Laminar flow behind a two-dimensional grid. *Mathematical Proceedings of the Cambridge Philosophical Society*, 44(1):58–62, January 1948. ISSN 1469-8064, 0305-0041. doi: 10.1017/S0305004100023999.
- [46] Aditi S. Krishnapriyan, Amir Gholami, Shandian Zhe, Robert M. Kirby, and Michael W. Mahoney. Characterizing possible failure modes in physics-informed neural networks, November 2021.
- [47] Songming Liu, Chang Su, Jiachen Yao, Zhongkai Hao, Hang Su, Youjia Wu, and Jun Zhu. Preconditioning for Physics-Informed Neural Networks, February 2024.
- [48] Kent-Andre Mardal and Ragnar Winther. Preconditioning discretizations of systems of partial differential equations. *Numerical Linear Algebra with Applications*, 18(1):1–40, 2011. ISSN 1099-1506. doi: 10.1002/nla.716.
- [49] Charles C. Margossian. A Review of automatic differentiation and its efficient implementation. *WIREs Data Mining and Knowledge Discovery*, 9(4):e1305, July 2019. ISSN 1942-4787, 1942-4795. doi: 10.1002/WIDM.1305.
- [50] James Martens. New insights and perspectives on the natural gradient method, September 2020.
- [51] Per-Gunnar Martinsson and Joel A. Tropp. Randomized numerical linear algebra: Foundations and algorithms. *Acta Numerica*, 29:403–572, May 2020. ISSN 0962-4929, 1474-0508. doi: 10.1017/S0962492920000021.
- [52] Patrick Kofod Mogensen and Asbjørn Nilsen Riseth. Optim: A mathematical optimization package for Julia. *Journal of Open Source Software*, 3(24):615, 2018. doi: 10.21105/joss.00615.
- [53] Riley Murray, James Demmel, Michael W. Mahoney, N. Benjamin Erichson, Maksim Melnichenko, Osman Asif Malik, Laura Grigori, Piotr Luszczek, Michał Dereziński, Miles E. Lopes, Tianyu Liang, Hengrui Luo, and Jack Dongarra. Randomized Numerical Linear Algebra : A Perspective on the Field With an Eye to Software, April 2023.
- [54] Johannes Müller and Marius Zeinhofer. Achieving High Accuracy with PINNs via Energy Natural Gradients, August 2023.
- [55] Johannes Müller and Marius Zeinhofer. Position: Optimization in SciML Should Employ the Function Space Geometry, May 2024.
- [56] Levon Nurbekyan, Wanzhou Lei, and Yunan Yang. Efficient Natural Gradient Descent Methods for Large-Scale PDE-Based Optimization Problems. *SIAM Journal on Scientific Computing*, 45(4):A1621–A1655, August 2023. ISSN 1064-8275. doi: 10.1137/22M1477805.
- [57] Avik Pal. Lux: Explicit Parameterization of Deep Neural Networks in Julia, April 2023.
- [58] David Pfau, James S. Spencer, Alexander G. D. G. Matthews, and W. M. C. Foulkes. Ab initio solution of the many-electron Schrödinger equation with deep neural networks. *Physical Review Research*, 2(3):033429, September 2020. doi: 10.1103/PhysRevResearch.2.033429.
- [59] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, February 2019. ISSN 0021-9991. doi: 10.1016/j.jcp.2018.10.045.
- [60] Pratik Rathore, Weimu Lei, Zachary Frangella, Lu Lu, and Madeleine Udell. Challenges in Training PINNs: A Loss Landscape Perspective, June 2024.

- [61] Jon A. Rivera, Jamie M. Taylor, Ángel J. Omella, and David Pardo. On quadrature rules for solving Partial Differential Equations using Neural Networks. *Computer Methods in Applied Mechanics and Engineering*, 393:114710, April 2022. ISSN 0045-7825. doi: 10.1016/j.cma.2022.114710.
- [62] Tim De Ryck and Siddhartha Mishra. Numerical analysis of physics-informed neural networks and related models in physics-informed machine learning. *Acta Numerica*, 33:633–713, July 2024. ISSN 0962-4929, 1474-0508. doi: 10.1017/S0962492923000089.
- [63] Justin Sirignano and Konstantinos Spiliopoulos. DGM: A deep learning algorithm for solving partial differential equations. *Journal of Computational Physics*, 375:1339–1364, December 2018. ISSN 0021-9991. doi: 10.1016/j.jcp.2018.08.029.
- [64] Jamie M. Taylor and David Pardo. Stochastic Quadrature Rules for Solving PDEs using Neural Networks, April 2025.
- [65] Madeleine Udell and Zachary Frangella. Randomized Numerical Linear Algebra for Optimization. 2023.
- [66] Remco van der Meer, Cornelis W. Oosterlee, and Anastasia Borovykh. Optimally weighted loss functions for solving PDEs with Neural Networks. *Journal of Computational and Applied Mathematics*, 405:113887, May 2022. ISSN 0377-0427. doi: 10.1016/j.cam.2021.113887.
- [67] Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, January 2021. ISSN 1064-8275. doi: 10.1137/20M1318043.
- [68] Sifan Wang, Xinling Yu, and Paris Perdikaris. When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, January 2022. ISSN 0021-9991. doi: 10.1016/j.jcp.2021.110768.
- [69] Sifan Wang, Shyam Sankaran, and Paris Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421:116813, March 2024. ISSN 0045-7825. doi: 10.1016/j.cma.2024.116813.
- [70] Stephen J. Wright and Jorge Nocedal. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer New York, 2006. ISBN 978-0-387-30303-1. doi: 10.1007/978-0-387-40065-5.
- [71] Chenxi Wu, Min Zhu, Qinyang Tan, Yadhu Kartha, and Lu Lu. A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 403:115671, January 2023. ISSN 0045-7825. doi: 10.1016/j.cma.2022.115671.
- [72] Marius Zeinhofer, Rami Masri, and Kent-André Mardal. A Unified Framework for the Error Analysis of Physics-Informed Neural Networks, March 2024.
- [73] Qi Zeng, Yash Kothari, Spencer H. Bryngelson, and Florian Schäfer. Competitive Physics Informed Networks, October 2022.

A Pseudocode of Randomized Nyström Approximation

B An extended discussion on the choice of the metric

In this Appendix section, we further elaborate on the selection of the metric discussed in Section 3.1, providing additional details and extending the discussion.

It can be shown that NGD corresponds to Riemannian gradient descent [14] on the manifold $(\mathcal{M}, g)$, up to the g -orthogonal projection of the Riemannian gradient onto the model’s generalized tangent space $T_{\theta}\mathcal{N}$, and a second-order correction; see [55, Theorem 1]. More precisely, the NGD update $\theta_{k+1} = \theta_k - \alpha_k \mathbf{G}(\theta_k)^\dagger \nabla L(\theta_k)$ satisfies

$$u_{\theta_{k+1}} = u_{\theta_k} - \alpha_k \Pi_{\theta_k}^g (\text{grad}^g E(u_{\theta_k})) + \varepsilon_k,$$

Algorithm 2 Randomized Nyström Approximation [26, 51]

Require: PSD matrix $\mathbf{G} \in \mathbb{R}^{p \times p}$ (as a function $\mathbf{v} \mapsto \mathbf{G}\mathbf{v}$), rank ℓ

Ensure: Nyström approximation $\hat{\mathbf{G}}_{\text{nys}} = \mathbf{U}\hat{\mathbf{\Lambda}}\mathbf{U}^\top$

- | | |
|--|------------------------------------|
| 1: $\mathbf{\Omega} = \text{randn}(p, \ell)$ | ▷ Gaussian test matrix |
| 2: $\mathbf{\Omega} = \text{qr}(\mathbf{\Omega}, 0)$ | ▷ Thin QR decomposition |
| 3: $\mathbf{Y} = \mathbf{G}\mathbf{\Omega}$ | ▷ ℓ matvecs with $\mathbf{G}$ |
| 4: $\nu = \text{eps}(\text{norm}(\mathbf{Y}, \text{'fro'}))$ | ▷ Compute shift |
| 5: $\mathbf{Y}_\nu = \mathbf{Y} + \nu\mathbf{\Omega}$ | ▷ Shift for stability |
| 6: $\mathbf{C} = \text{chol}(\mathbf{\Omega}^\top \mathbf{Y}_\nu)$ | |
| 7: $\mathbf{B} = \mathbf{Y}_\nu / \mathbf{C}$ | ▷ Triangular solve |
| 8: $[\mathbf{U}, \mathbf{\Sigma}, \sim] = \text{svd}(\mathbf{B}, 0)$ | ▷ Thin SVD |
| 9: $\hat{\mathbf{\Lambda}} = \max\{0, \mathbf{\Sigma}^2 - \nu\mathbf{I}\}$ | ▷ Remove shift |
-

where $\Pi_{\theta_k}^g$ denotes the g -orthogonal projection onto the generalized tangent space $\text{T}_{\theta_k}\mathcal{N}$, and $\varepsilon_k = \mathcal{O}(\alpha_k^2 \|\mathbf{G}(\theta_k)^\top \nabla L(\theta_k)\|^2)$. A principled strategy for selecting the metric g is thus to adopt criteria from Riemannian optimization, choosing g to reduce the condition number of the Riemannian Hessian [1, 14]. Although convergence may differ from standard Riemannian gradient descent due to the projection onto $\text{T}_{\theta}\mathcal{N}$, whose effect depends on the geometry of $\mathcal{N}$ and the parametrization, this strategy can be expected to remain effective if the ansatz $\mathcal{N}$ is sufficiently expressive.

B.1 Linear problems

Setting B.1. Let V, W be Hilbert spaces, and let $\mathcal{A} : V \rightarrow W'$ be a linear isomorphism. Given $f \in W'$ we aim at solving

$$\mathcal{A}u = f \in W'. \quad (15)$$

A natural approach is to consider the least-squares problem

$$\inf_{u \in V} E(u) := \frac{1}{2} \|\mathcal{A}u - f\|_{W'}^2. \quad (16)$$

Choosing on $\mathcal{M} = V$ the (point-independent) metric

$$g(v, w) := (\mathcal{A}v, \mathcal{A}w)_{W'}, \quad (17)$$

yields $\text{Hess}^g E = \text{Id}$. Riemannian gradient descent on (V, g) thus converges in a single iteration, as the Riemannian gradient is given by $\text{grad}^g E(u) = u - \mathcal{A}^{-1}f$, i.e., computing it is equivalent to solving (15). NGD does not require applying $\mathcal{A}^{-1}$: computing the Gramian $[\mathbf{G}(\theta)]_{ij} = (\mathcal{A}\partial_{\theta_i} u_\theta, \mathcal{A}\partial_{\theta_j} u_\theta)_{W'}$ involves only applications of $\mathcal{A}$.

Remark B.2. We formulate the linear problem from V to the dual of W because one of our target applications involves variational formulations of PDEs, which naturally take the form (15). For PINNs, which are based on strong formulations, W is typically a (product of) L^2 spaces, so that $W' \cong W$ and the formulation recovers that of [72], which does not employ the dual.

Remark B.3. In Setting B.1, it is typically straightforward to approximate numerically the inner product $(\cdot, \cdot)_W$ and the duality pairing $\langle \cdot, \cdot \rangle_{W' \times W}$, but not the dual inner product $(\cdot, \cdot)_{W'}$. Let $\mathcal{R}_W : W \rightarrow W'$ denote the Riesz map, defined by

$$\langle \mathcal{R}_W v, w \rangle_{W' \times W} = (v, w)_W \quad \forall v, w \in W.$$

Then, for $\ell, h \in W'$, we have

$$(\ell, h)_{W'} = \langle \ell, \mathcal{R}_W^{-1} h \rangle_{W' \times W}.$$

For example, when $W = H_0^1(\Omega)$, the dual space is $W' = H^{-1}(\Omega)$, and $\mathcal{R}_W = \Delta$, the Laplace operator. Computing the dual inner product thus requires applying $(-\Delta^{-1})$, i.e., solving a (discretized) Laplace problem. In contrast, for $W = L^2(\Omega)$, one has $W' \cong W$ and $\mathcal{R}_W = \text{Id}$.

Symmetric positive definite linear operators If $V = W$ and $\mathcal{A}$ is symmetric positive definite, one can endow V with the $\mathcal{A}$ -induced inner product

$$(v, w)_{\mathcal{A}} = \langle \mathcal{A}v, w \rangle_{V' \times V} \quad \forall v, w \in V.$$

This corresponds to choosing $\mathcal{R}_V = \mathcal{A}$, and the loss (16) becomes

$$E(u) = \frac{1}{2} \langle \mathcal{A}u, u \rangle_{V' \times V} - \langle f, u \rangle_{V' \times V} + \text{const}, \quad (18)$$

where the constant is irrelevant for gradient-based optimization. This is precisely the variational form used in the Deep Ritz method [25]. The corresponding natural metric is $g = (\cdot, \cdot)_{\mathcal{A}}$.

B.2 Nonlinear problems

For general nonlinear problems, there is no canonical choice of metric as in the linear case. In some instances, the structure of the problem suggests multiple metrics; the choice of the metric impacts the convergence and the computational cost of the optimization schemes. Examples include Variational Monte Carlo (VMC) [36, 55, 58] or Sobolev gradient flows [17, 35]. Below we outline some general strategies that can be used in nonlinear problems and are employed in our numerical experiments.

Newton's method

Setting B.4. Let $(\mathcal{M}, a)$ be a Riemannian manifold and $f : \mathcal{M} \rightarrow \mathbb{R}$ a smooth function. We aim to solve $\inf_{u \in \mathcal{M}} E(u)$.

If $\text{Hess}^a E(u)$ is symmetric positive definite for all $u \in \mathcal{M}$, a natural metric leading to Riemannian Newton's method [14] is

$$g_u(v, w) = \text{Hess}^a E(u)[v, w]. \quad (19)$$

In the case $(\mathcal{M}, a) = (V, (\cdot, \cdot)_V)$ for a Hilbert space V , this yields the Energy Natural Gradient Descent (ENGd) method [54, 55]. Note that to interpret ENGd as a Natural Gradient Descent, it is necessary that $\text{Hess}^a E(u)$ is SPD for all $u \in \mathcal{M}$, as otherwise (19) does not define a valid metric.

Nonlinear least-squares problem: Gauss-Newton

Setting B.5. Let V, W be Hilbert spaces, and let $\mathcal{R} : V \rightarrow W$ be a (possibly nonlinear) residual. We aim at finding $u \in V$ such that $\mathcal{R}(u) = 0$.

A natural approach is to minimize the nonlinear least-squares functional

$$\inf_{u \in V} E(u) := \inf_{u \in V} \frac{1}{2} \|\mathcal{R}(u)\|_W^2. \quad (20)$$

The Hessian of f reads

$$D^2 E(u)[v, w] = (D\mathcal{R}(u)[v], D\mathcal{R}(u)[w])_W + (\mathcal{R}(u), D^2 \mathcal{R}(u)[v, w])_W,$$

and a symmetric positive semidefinite approximation is given by the Gauss-Newton method [70]

$$g_u(v, w) = (D\mathcal{R}(u)[v], D\mathcal{R}(u)[w])_W. \quad (21)$$

This choice of the metric leads the Gauss-Newton NGD method [33, 38]. Note that g_u is, in general, only semidefinite. While this does not pose numerical issues when using the pseudoinverse or damping, the positive definiteness of g_u is necessary to interpret it as a Riemannian metric. Whether this condition holds depends on the structure of the underlying nonlinear PDE.

C Proof of Theorem 3.2

Assumption 3.1. Let $\Omega \subset \mathbb{R}^d$ be a domain, and let $\mathcal{M} \subseteq V$, where V is a Hilbert space of functions on Ω . Assume that the metric g_u takes the form

$$g_u(v, w) = \int_{\Omega} \mathcal{F}_u v(\mathbf{x}) \cdot \mathcal{F}_u w(\mathbf{x}) \varphi_u(\mathbf{x}) \, d\mathbf{x}, \quad (10)$$

for all $v, w \in T_u \mathcal{M} \subseteq V$, where $\mathcal{F}_u : V \rightarrow L^2(\Omega; \varphi)^k$ is linear and continuous.

Theorem 3.2. Let Assumption 3.1 hold, and let $\text{sg} : \mathbb{R}^p \rightarrow \mathbb{R}^p$ denote the stop-gradient operator, formally defined by $\text{sg}(\boldsymbol{\theta}) = \boldsymbol{\theta}$ and $\nabla \text{sg}(\boldsymbol{\theta}) = \mathbf{0}$. If the integral (10) is approximated using a quadrature rule with points $\{\mathbf{x}_r\}_{r=1}^q$ and weights $\{w_r\}_{r=1}^q$, then

$$\mathbf{G}(\boldsymbol{\theta}) = \mathbf{J}_\theta \mathbf{F}(\boldsymbol{\theta})^\top \begin{bmatrix} w_1 \varphi_{u_\theta}(\mathbf{x}_1) \mathbf{I}_k & & \\ & \ddots & \\ & & w_q \varphi_{u_\theta}(\mathbf{x}_q) \mathbf{I}_k \end{bmatrix} \mathbf{J}_\theta \mathbf{F}(\boldsymbol{\theta}), \quad \mathbf{F}(\boldsymbol{\theta}) = \begin{bmatrix} \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_\theta(\mathbf{x}_1) \\ \vdots \\ \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_\theta(\mathbf{x}_q) \end{bmatrix}, \quad (11)$$

where $\mathbf{J}_\theta$ denotes the Jacobian with respect to $\boldsymbol{\theta}$.

Proof. Let $\mathbf{e}_h \in \mathbb{R}^p$ denote the h -th vector of the canonical basis, i.e., the vector that has a 1 in the h -th entry and 0 in all other entries. Using the linearity and continuity of $\mathcal{F}_{u_\theta}$,

$$\begin{aligned} \mathcal{F}_{u_\theta} \partial_{\theta_h} u_\theta &= \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} \partial_{\theta_h} u_\theta = \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}}^k \left(\lim_{t \rightarrow 0} \frac{u_{\boldsymbol{\theta} + t \mathbf{e}_h} - u_\theta}{t} \right) \\ &= \lim_{t \rightarrow 0} \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} \left(\frac{u_{\boldsymbol{\theta} + t \mathbf{e}_h} - u_\theta}{t} \right) = \lim_{t \rightarrow 0} \frac{\mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_{\boldsymbol{\theta} + t \mathbf{e}_h} - \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_\theta}{t} \\ &= \partial_{\theta_h} \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_\theta. \end{aligned}$$

Note that the use of the stop-gradient operator is crucial to ensure that we do not differentiate $\mathcal{F}_{u_\theta}$ with respect to $\boldsymbol{\theta}$. Hence,

$$\begin{aligned} [\mathbf{G}(\boldsymbol{\theta})]_{ij} &= g_{u_\theta}(\partial_{\theta_i} u_\theta, \partial_{\theta_j} u_\theta) = \int_{\Omega} \mathcal{F}_{u_\theta} \partial_{\theta_i} u_\theta \cdot \mathcal{F}_{u_\theta} \partial_{\theta_j} u_\theta(\mathbf{x}) \varphi_{u_\theta} d\mathbf{x} \\ &= \int_{\Omega} \partial_{\theta_i} \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_\theta \cdot \partial_{\theta_j} \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_\theta \varphi_{u_\theta} d\mathbf{x}, \end{aligned}$$

or, in matrix form,

$$\mathbf{G}(\boldsymbol{\theta}) = (\mathbf{D}_\theta \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_\theta)^* \mathbf{D}_\theta \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_\theta,$$

where the Hilbert adjoint $(\mathbf{D}_\theta \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_\theta)^*$ is taken with respect to the inner product on $L^2(\Omega, \varphi)^k$, and we identify a matrix with the induced linear map.

Applying quadrature with nodes $\{\mathbf{x}_r\}_{r=1}^q$ and weights $\{w_r\}_{r=1}^q$, we get

$$[\mathbf{G}(\boldsymbol{\theta})]_{ij} = \sum_{r=1}^q w_r \partial_{\theta_i} \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_\theta(\mathbf{x}_r) \cdot \partial_{\theta_j} \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_\theta(\mathbf{x}_r) \varphi_{u_\theta}(\mathbf{x}_r)$$

and thus, in matrix form:

$$\begin{aligned} \mathbf{G}(\boldsymbol{\theta}) &= \sum_{r=1}^q w_r \varphi_{u_\theta}(\mathbf{x}_r) (\mathbf{J}_\theta \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_\theta(\mathbf{x}_r))^\top \mathbf{J}_\theta \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}} u_\theta(\mathbf{x}_r) \\ &= \mathbf{J}_\theta \mathbf{F}(\boldsymbol{\theta})^\top \begin{bmatrix} w_1 \varphi_{u_\theta}(\mathbf{x}_1) \mathbf{I}_k & & \\ & \ddots & \\ & & w_q \varphi_{u_\theta}(\mathbf{x}_q) \mathbf{I}_k \end{bmatrix} \mathbf{J}_\theta \mathbf{F}(\boldsymbol{\theta}). \end{aligned}$$

□

Remark C.1. More in general, it can be shown that if the metric g_u takes the form

$$g_u(v, w) = \int_{\Omega} \mathcal{F}_u^1 v(\mathbf{x}) \cdot \mathcal{F}_u^2 w(\mathbf{x}) \varphi_u(\mathbf{x}) d\mathbf{x}, \quad (22)$$

where $\mathcal{F}_u^i : V \rightarrow L^2(\Omega; \varphi)^k$ are linear and continuous for $i = 1, 2$, and the integral is approximated using a quadrature rule with points $\{\mathbf{x}_r\}_{r=1}^q$ and weights $\{w_r\}_{r=1}^q$, then

$$\mathbf{G}(\boldsymbol{\theta}) = \mathbf{J}_\theta \mathbf{F}_1(\boldsymbol{\theta})^\top \begin{bmatrix} w_1 \varphi_{u_\theta}(\mathbf{x}_1) \mathbf{I}_k & & \\ & \ddots & \\ & & w_q \varphi_{u_\theta}(\mathbf{x}_q) \mathbf{I}_k \end{bmatrix} \mathbf{J}_\theta \mathbf{F}_2(\boldsymbol{\theta}), \quad \mathbf{F}_i(\boldsymbol{\theta}) = \begin{bmatrix} \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}}^i u_\theta(\mathbf{x}_1) \\ \vdots \\ \mathcal{F}_{u_{\text{sg}(\boldsymbol{\theta})}}^i u_\theta(\mathbf{x}_q) \end{bmatrix}. \quad (23)$$

D Modification of `jaxopt.BFGS`

We refer to the JAXopt implementation of the BFGS algorithm in https://github.com/google/jaxopt/blob/main/jaxopt/_src/bfgs.py (commit c818497). Using the notation from the implementation, lines 240-250 compute the BFGS inverse Hessian update [70] as:

$$\mathbf{H}_{k+1} = (\mathbf{I} - \rho \mathbf{s} \mathbf{y}^\top) \mathbf{H}_k (\mathbf{I} - \rho \mathbf{y} \mathbf{s}^\top) + \rho \mathbf{s} \mathbf{s}^\top, \quad \rho = \frac{1}{\mathbf{s}^\top \mathbf{y}}.$$

This involves outer products $\mathbf{s} \mathbf{y}^\top$ and $\mathbf{s} \mathbf{s}^\top$ and two matrix-matrix multiplications. We replace it with a more efficient formulation that avoids full matrix-matrix products, using vector-matrix operations and rank-one updates:

$$\mathbf{H}_{k+1} = \mathbf{H}_k + [\rho + \rho^2 \mathbf{y}^\top (\mathbf{H}_k \mathbf{y})] \mathbf{s} \mathbf{s}^\top - \rho [(\mathbf{H}_k \mathbf{y}) \mathbf{s}^\top + \mathbf{s} (\mathbf{H}_k \mathbf{y})^\top].$$

This reformulation eliminates matrix-matrix multiplications, resulting in significant speedups while maintaining numerical stability and equivalence to the original update.

E Additional resources for the numerical experiments in Section 5

We begin with PDEs in strong form discretized via Physics-Informed Neural Networks (PINNs) [19, 59]. In all PINNs experiments, we set in Algorithm 1 the parameters $\ell_0 = 10$, $\ell_{\max} = 500$, $\gamma = p$, $\text{maxit} = 20$, and $\kappa = 0.1$. For NGD and NGD with CG, we use $\mu = \min(10^{-5}, L(\boldsymbol{\theta}))$ as suggested in [72], from which most experiments are adapted. To enforce stronger regularization in early iterations, in NyströmNGD we take the regularization parameter as $\mu = \max(p \cdot \hat{\lambda}_1 \cdot \epsilon_{\text{mach}}, 10^{-4} \cdot L(\boldsymbol{\theta})^2)$. Errors are always computed using an order of magnitude more quadrature points than in training, sampled independently. Random integration points sampled once and fixed throughout training.

E.1 Poisson problem in 3D

We consider the Poisson problem on the 3D unit cube

$$\begin{cases} -\Delta u = f & \Omega = (0, 1)^3, \\ u = g_D & \partial\Omega, \end{cases} \quad (24)$$

with exact solution $u(x, y, z) = \sin(\pi x) \sin(\pi y) \sin(\pi z)$ as in [72]. Problem (24) is linear and we consider it in strong form. Using the notation of Appendix B.1, the function spaces are therefore $V = H^2(\Omega)$ and $W = L^2(\Omega) \times L^2(\partial\Omega) \cong W'$, with operator

$$\mathcal{A} : H^2(\Omega) \rightarrow L^2(\Omega) \times L^2(\partial\Omega), \quad \mathcal{A}u = (-\Delta u, u|_{\partial\Omega}).$$

The corresponding least-squares loss is

$$E(u) = \frac{1}{2} \|\Delta u + f\|_{L^2(\Omega)}^2 + \frac{1}{2} \|u - g_D\|_{L^2(\partial\Omega)}^2, \quad (25)$$

and the induced metric reads

$$g(u, v) = (\Delta u, \Delta v)_{L^2(\Omega)} + (u, v)_{L^2(\partial\Omega)} = \frac{1}{2} \int_{\Omega} \Delta u \Delta v \, d\mathbf{x} + \frac{1}{2} \int_{\partial\Omega} u v \, d\mathbf{s}. \quad (26)$$

To discretize problem (24) using a PINN formulation, we take $u = u_{\boldsymbol{\theta}}$ a neural network and the loss function $L(\boldsymbol{\theta}) = E(u_{\boldsymbol{\theta}})$ is approximated using MonteCarlo quadrature.

Remark E.1. The metric (26) is coercive only in the $H^{1/2}(\Omega)$ norm, but not in $H^2(\Omega)$. This is due to the use of the $L^2(\partial\Omega)$ norm for the Dirichlet boundary term. Coercivity in $H^2(\Omega)$ can be restored by enforcing the boundary conditions strongly [72, Remark 6], or by penalizing in the $H^{1/2}(\partial\Omega)$ norm [13]. However, in line with standard PINNs practice, we retain the $L^2(\partial\Omega)$ penalty for simplicity.

Computational details We use a fully connected neural network $u_{\boldsymbol{\theta}}$ with tanh activation and two hidden layers of width 32. Integrals in the loss and metric are discretized via Monte Carlo quadrature with $N_{\Omega} = 3000$ points uniformly sampled in the interior Ω and $N_{\partial\Omega} = 800$ on the boundary $\partial\Omega$. We run NGD-based methods for 500 iterations and BFGS for 6×10^4 iterations.

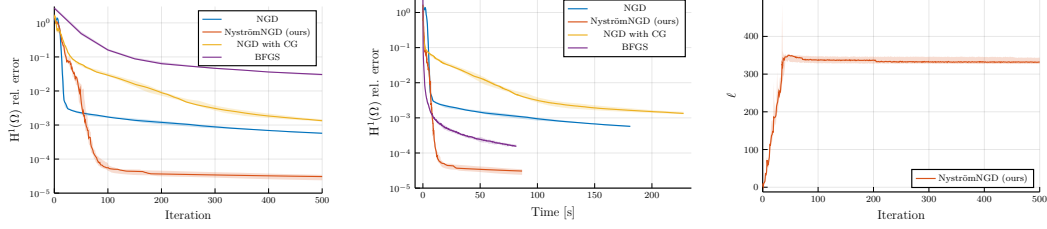


Figure 6: Poisson equation in 3D discretized via PINNs. From left to right: relative $H^1(\Omega)$ error vs. iterations (first 500 iterations), relative $H^1(\Omega)$ error vs. wall-clock time, and Nyström rank parameter ℓ vs. iterations, with $\ell_{\max} = 500$.

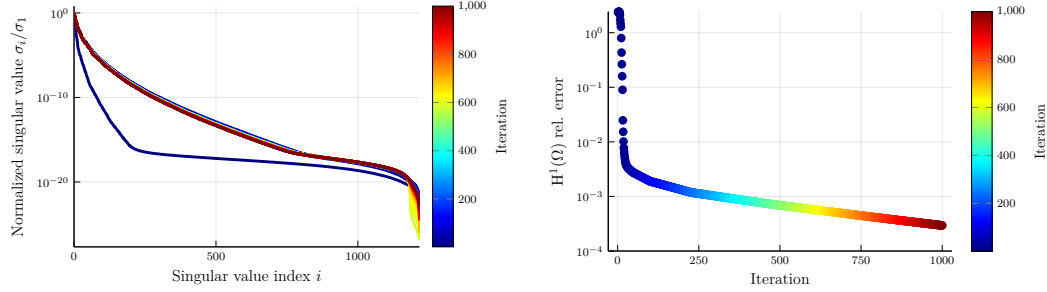


Figure 7: Poisson equation in 3D discretized via PINNs and trained with NGD using SVD-based pseudoinverse. On the left, the decay of the normalized singular values $\{\sigma_i/\sigma_1\}_{i=1}^p$ of the Gramian across iterations, with each color representing a different iteration. On the right, the relative $H^1(\Omega)$ error vs. iterations.

E.2 Heat equation in 3+1D

We consider the heat equation

$$\begin{cases} \partial_t u - \Delta u = f & I \times \Omega = (0, 1) \times (0, 1)^3, \\ u = g_D & I \times \partial\Omega, \\ u(0, \cdot) = u_0 & \Omega, \end{cases} \quad (27)$$

with exact solution $u(t, x, y, z) = (\cos(\pi x) + \cos(\pi y) + \cos(\pi z)) e^{-\pi^2 t/4}$ as in [72]. We consider Equation (27) in strong form, with function spaces $V = L^2(I, H^2(\Omega)) \cap H^1(I, L^2(\Omega))$, $W = L^2(I, L^2(\Omega)) \times L^2(I, L^2(\partial\Omega)) \times L^2(\Omega)$, and linear operator

$$\mathcal{A}u = (\partial_t u - \Delta u, u|_{I \times \partial\Omega}, u(0, \cdot)).$$

The corresponding least squares loss is

$$E(u) = \frac{1}{2} \|\partial_t u - \Delta u - f\|_{L^2(I, L^2(\Omega))}^2 + \frac{1}{2} \|u - g_D\|_{L^2(I, L^2(\partial\Omega))}^2 + \frac{1}{2} \|u(0, \cdot) - u_0\|_{L^2(\Omega)}^2, \quad (28)$$

with induced metric

$$g(u, v) = (\partial_t u - \Delta u, \partial_t v - \Delta v)_{L^2(I, L^2(\Omega))} + (u, v)_{L^2(I, L^2(\Omega))} + (u(0, \cdot), v(0, \cdot))_{L^2(\Omega)}. \quad (29)$$

Considerations on the coercivity of the metric similar to Remark E.1 apply here. For details, we refer to [72, Section 4.5]. To discretize problem (27) we take $u = u_\theta$ a neural network and approximate $L(\theta) = E(u_\theta)$ using MonteCarlo quadrature.

Computational details We use a fully connected neural network u_θ with tanh activation and two hidden layers of width 32. Integrals in the loss and metric are discretized via Monte Carlo quadrature with $N_{I \times \Omega} = 3000$ points uniformly sampled in the interior $I \times \Omega$ and $N_\Gamma = 800$ on the spatio-temporal boundary $\Gamma = (I \times \partial\Omega) \cup (\{0\} \times \Omega)$. We run NGD-based methods for 1000 iterations and BFGS for 6×10^4 iterations.

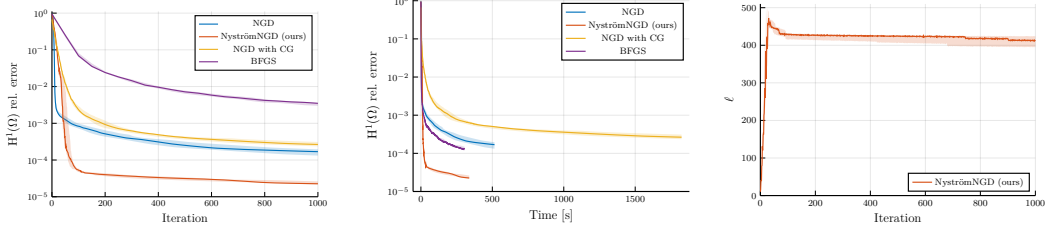


Figure 8: Heat equation in 3+1D discretized via PINNs. From left to right: relative $H^1(\Omega)$ error vs. iterations (first 1000 iterations), relative $H^1(\Omega)$ error vs. wall-clock time, and Nyström rank parameter ℓ vs. iterations, with $\ell_{\max} = 500$.

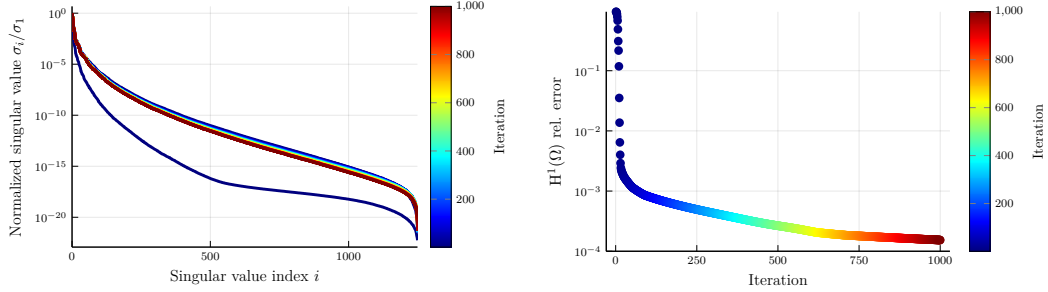


Figure 9: Heat equation in 3+1D discretized via PINNs and trained with NGD using SVD-based pseudoinverse. On the left, the decay of the normalized singular values $\{\sigma_i/\sigma_1\}_{i=1}^p$ of the Gramian across iterations, with each color representing a different iteration. On the right, the relative $H^1(\Omega)$ error vs. iterations.

E.3 Kovasznay Flow (2D steady state Navier–Stokes flow)

We consider the two-dimensional steady Navier–Stokes flow as originally described in [45], with Reynolds number $\text{Re} = 40$, following the setup in [38, Section 4.1]. The system of PDEs reads

$$\begin{cases} -\nu \Delta u + (u \cdot \nabla)u + \nabla p = f & \Omega = (-\frac{1}{2}, 1) \times (-\frac{1}{2}, \frac{3}{2}), \\ \nabla \cdot u = 0 & \Omega, \\ u = g_D & \partial\Omega, \end{cases} \quad (30)$$

with exact solution

$$\begin{aligned} u(x, y) &= \begin{bmatrix} 1 - e^{\lambda x} \cos(2\pi y) \\ \frac{\lambda}{2\pi} e^{\lambda x} \sin(2\pi y) \end{bmatrix}, \\ p(x, y) &= \frac{1}{2}(1 - e^{2\lambda x}) - c, \end{aligned}$$

where $\lambda = \frac{1}{2\nu} - \sqrt{\frac{1}{4\nu^2} + 4\pi^2}$, $\nu = \frac{1}{\text{Re}} = \frac{1}{40}$, and c is chosen such that $\int_{\Omega} p = 0$. Following [38], we consider the function spaces $V = H^2(\Omega) \times \{p \in H^1(\Omega) : \int_{\Omega} p = 0\}$ and $W = L^2(\Omega)^2 \times L^2(\Omega) \times L^2(\partial\Omega) \cong W'$. We write problem (30) in strong form as a nonlinear least squares problem with residual

$$\mathcal{R}(u, p) = \begin{bmatrix} -\nu \Delta u + (u \cdot \nabla)u + \nabla p - f \\ \nabla \cdot u \\ u - g_D \end{bmatrix} \quad (31)$$

and corresponding energy functional

$$E(u, p) = \frac{1}{2} \|\nu \Delta u + (u \cdot \nabla)u + \nabla p - f\|_{L^2(\Omega)^2}^2 + \frac{1}{2} \|\nabla \cdot u\|_{L^2(\Omega)}^2 + \frac{1}{2} \|u - g_D\|_{L^2(\partial\Omega)}^2. \quad (32)$$

As a metric we consider the Gauss–Newton metric (9). We refer to [38] for a discussion on coercivity. To discretize (30), we take $(u, p) = (u_{\theta_1}, p_{\theta_2})$ two neural networks with weights combined into a single vector $\theta = [\theta_1, \theta_2]$, and approximate $\tilde{L}(\theta) = E(u_{\theta_1}, p_{\theta_2})$ using Monte Carlo quadrature.

Computational details We employ fully connected neural networks u_{θ_1} and p_{θ_2} with tanh activation and two hidden layers of width 32. Integrals in the loss and metric are discretized via Monte Carlo quadrature with $N_{\Omega} = 3000$ points uniformly sampled in the interior Ω and $N_{\partial\Omega} = 800$ on the boundary $\partial\Omega$. We run NGD-based methods for 1000 iterations and BFGS for 1.2×10^5 iterations.

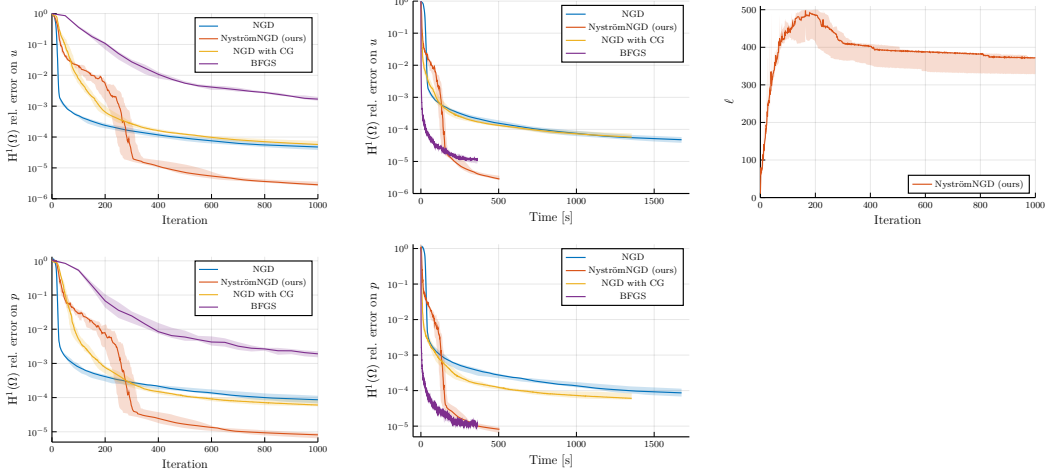


Figure 10: Kovasny Flow discretized via PINNs. Top row, from left to right: relative $H^1(\Omega)$ error on u vs. iterations (first 1000 iterations), relative $H^1(\Omega)$ error on u vs. wall-clock time, and Nystrom rank parameter ℓ vs. iterations, with $\ell_{\max} = 500$. Bottom row, from left to right: relative $H^1(\Omega)$ error on p vs. iterations (first 1000 iterations), and relative $H^1(\Omega)$ error on p vs. wall-clock time.

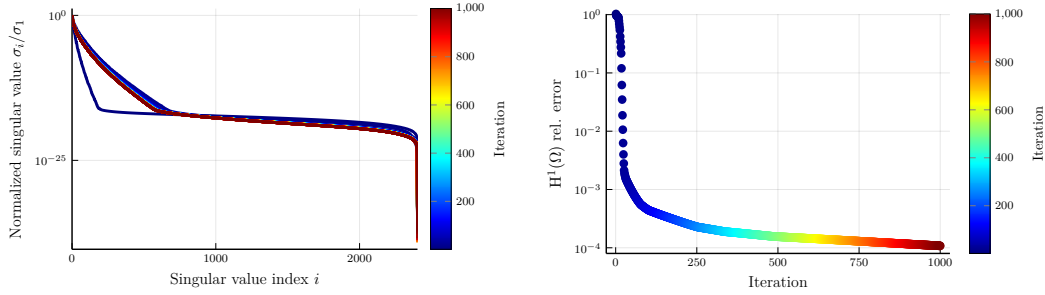


Figure 11: Kovasny Flow discretized via PINNs and trained with NGD using SVD-based pseudoinverse. On the left, the decay of the normalized singular values $\{\sigma_i/\sigma_1\}_{i=1}^p$ of the Gramian across iterations, with each color representing a different iteration. On the right, the relative $H^1(\Omega)$ error on u vs. iterations.

E.4 Source recovery problem for the Poisson equation in 3D

Given a penalization parameter $\eta > 0$ and interior observations $u_d \in L^2(\Omega)$, we consider the inverse problem of recovering both the solution u and the source term f from data by minimizing the Tikhonov-regularized objective:

$$\inf_{(u,f) \in H \times L^2(\Omega)} J(u, f) = \frac{1}{2} \|u - u_d\|_{L^2(\Omega)}^2 + \frac{\eta^2}{2} \|f\|_{L^2(\Omega)}^2, \quad (33)$$

subject to the Poisson PDE constraint

$$\begin{cases} -\Delta u = f & \text{in } \Omega = (0,1)^3, \\ u = g_D & \text{on } \partial\Omega, \end{cases} \quad (34)$$

where $H = \overline{C^\infty(\bar{\Omega})}^{\|\cdot\|_H}$ with $\|u\|_H = \|\Delta u\|_{L^2(\Omega)} + \|u\|_{L^2(\partial\Omega)}$. See [72, Section 4.7] for well-posedness and theoretical background. While the solution u in [72] yields $f = 0$, in this paper

we choose a more complex problem, where $u(x, y, z) = \sin(\pi x) \sin(\pi y) \sin(\pi z)$ and the resulting source term to be reconstructed is not identically zero.

We discretize problem (33–34) in strong form using a PINN formulation, where observations $u_d \in L^2(\Omega)$ are in practice replaced by N_{data} point measurements. The function spaces are $V = H \times L^2(\Omega)$, $W = L^2(\Omega) \times L^2(\Omega) \times L^2(\partial\Omega)L^2(\Omega)$, and the linear operator

$$\mathcal{A}u = (\Delta u + f, u, u|_{\partial\Omega}, \eta f).$$

The corresponding least squares loss is

$$E(u, f) = \frac{1}{2} \|\Delta u + f\|_{L^2(\Omega)}^2 + \frac{1}{2} \|u - g_D\|_{L^2(\partial\Omega)}^2 + \frac{1}{2} \|u - u_d\|_{L^2(\Omega)}^2 + \frac{\eta^2}{2} \|f\|_{L^2(\Omega)}^2, \quad (35)$$

with induced metric

$$g((u, f), (v, h)) = (\Delta u + f, \Delta v + h)_{L^2(\Omega)} + (u, v)_{L^2(\partial\Omega)} + (u, v)_{L^2(\Omega)} + \eta^2 (f, h)_{L^2(\Omega)}. \quad (36)$$

We take $(u, f) = (u_{\theta_1}, f_{\theta_2})$ two neural networks with weights combined into a single vector $\theta = [\theta_1, \theta_2]$, and approximate $L(\theta) = E(u_{\theta_1}, f_{\theta_2})$ using Monte Carlo quadrature and point observations of u_d . Although $\eta > 0$ is theoretically required to ensure the minimum of (35) exists in V , we set $\eta = 0$ in our numerical experiments.

Computational details We employ fully connected neural networks u_{θ_1} and f_{θ_2} with tanh activation and two hidden layers of width 32. The term $\frac{1}{2} \|u - u_d\|_{L^2(\Omega)}^2$ in the energy is discretized using $N_{\text{data}} = 40$ noiseless observations of the true solution at random points in Ω . The remaining integrals in the loss and metric are discretized via Monte Carlo quadrature with $N_{\Omega} = 3000$ points uniformly sampled in the interior Ω and $N_{\partial\Omega} = 800$ on the boundary $\partial\Omega$. We run NGD-based methods for 500 iterations and BFGS for 1.2×10^5 iterations.

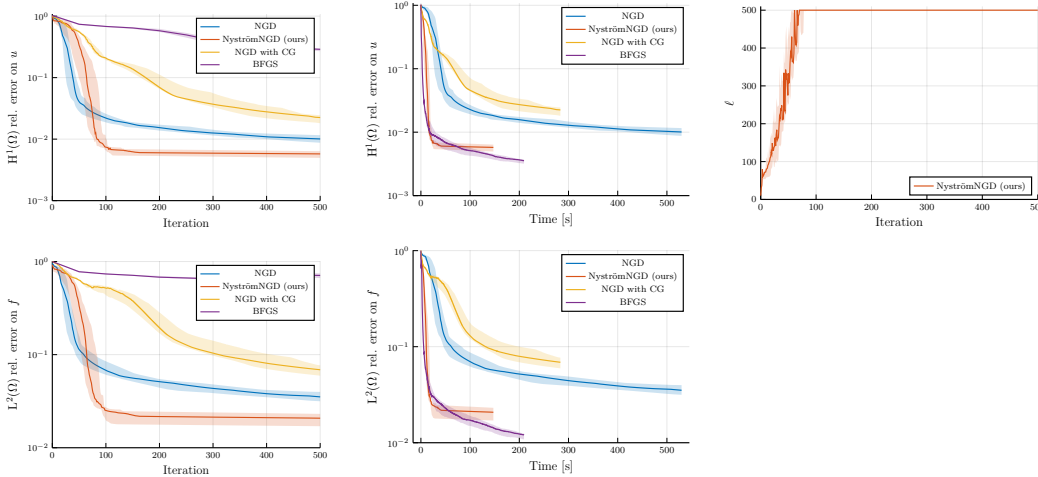


Figure 12: Source recovery problem for the Poisson equation in 3D discretized via PINNs. Top row, from left to right: relative $H^1(\Omega)$ error on u vs. iterations (first 500 iterations), relative $H^1(\Omega)$ error on u vs. wall-clock time, and Nyström rank parameter ℓ vs. iterations, with $\ell_{\max} = 500$. Bottom row, from left to right: relative $L^2(\Omega)$ error on f vs. iterations (first 500 iterations), and relative $L^2(\Omega)$ error on f vs. wall-clock time.

F Additional Numerical Experiments: Variational Formulations of PDEs

We now consider variational formulations of PDEs, where the solution is approximated by a neural network u_{θ} . We consider formulations closely related to Variational PINNs (VPINNs) [40, 41] and the Deep Ritz method [25]. In all examples, we interpolate the neural network onto a conforming finite element (FE) space, following [7]. This approach offers several advantages. First, Dirichlet boundary conditions can be approximately enforced by applying them to the FE interpolant using

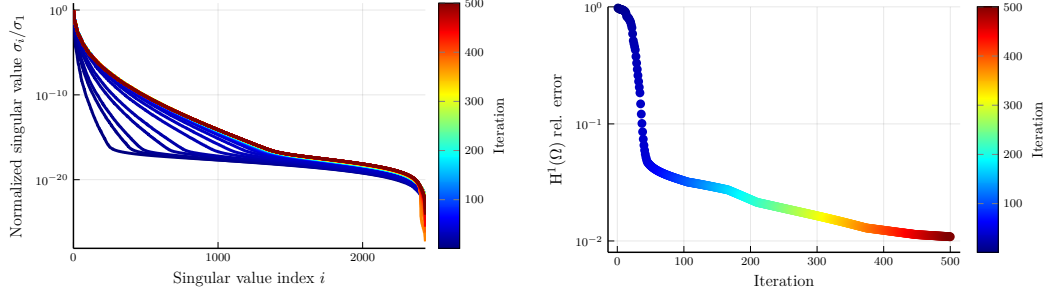


Figure 13: Source recovery problem for the Poisson equation in 3D discretized via PINNs and trained with NGD using SVD-based pseudoinverse. On the left, the decay of the normalized singular values $\{\sigma_i/\sigma_1\}_{i=1}^p$ of the Gramian across iterations, with each color representing a different iteration. On the right, the relative $H^1(\Omega)$ error on u vs. iterations.

standard FE techniques, avoiding the need to impose them directly on the neural network output [9, 10]. Second, it alleviates integration challenges, a known issue in neural network-based PDE solvers [61], especially in the Deep Ritz method [64]. Finally, spatial derivatives can be computed at the FE level, eliminating the need for automatic differentiation with respect to the input of the neural network. Further details are provided in [6, 7].

In all experiments involving variational formulations, we set the parameters in Algorithm 1 as follows: $\ell_0 = 10$, $\ell_{\max} = 500$, $\gamma = 30$, $\text{maxit} = 20$, and $\kappa = 0.1$. The neural networks are interpolated on a finite element space of degree $d = 4$ on a uniform quadrilateral mesh with mesh size $h = 1/30$, yielding approximately 14,500 degrees of freedom. Integrals in the loss and in the metric are evaluated by interpolation onto the FE space and Gauss-Jacobi quadrature of order $2d = 8$. Errors are computed with a Gauss-Jacobi quadrature rule of order $4d = 16$. We rely on the Julia packages Lux.jl [57] for neural networks, and Optim.jl [52] for optimization.

F.1 Poisson problem in 2D – Deep Ritz formulation

We consider the Poisson problem on the 2D unit square

$$\begin{cases} -\Delta u = f & \Omega = (0, 1)^2, \\ u = g_D & \partial\Omega, \end{cases} \quad (37)$$

where $f \in H^{-1}(\Omega)$ and $g_D \in H^{\frac{1}{2}}(\partial\Omega)$. As exact solution we take $u(x, y) = \sin(\pi x) \sin(\pi y)$.

All equalities in (37) are understood in the variational sense. That is, given a lifting $\tilde{g}_D \in H^1(\Omega)$ such that $\tilde{g}_D|_{\partial\Omega} = g_D$, we seek $u_0 \in H_0^1(\Omega)$ satisfying

$$a(u_0, v) = \int_{\Omega} f v \, dx - a(\tilde{g}_D, v) \quad \forall v \in H_0^1(\Omega), \quad \text{where} \quad a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v \, dx. \quad (38)$$

The solution is then obtained as $u = u_0 + \tilde{g}_D$. The spaces for the discretization are therefore $V = W = H_0^1(\Omega)$, and the linear operator is given by

$$\mathcal{A} : H_0^1(\Omega) \rightarrow H^{-1}(\Omega), \quad \langle \mathcal{A}u, v \rangle_{H^{-1} \times H_0^1} = a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v \, dx.$$

Since a is a symmetric and coercive bilinear form on $V \times V$, we can employ the Deep Ritz formulation and consider the minimization problem

$$\min_{u_0 \in H_0^1(\Omega)} E(u_0) := \int_{\Omega} \frac{1}{2} \|\nabla u_0\|^2 - f u_0 + \nabla \tilde{g}_D \cdot \nabla u_0 \, dx, \quad (39)$$

with corresponding metric is $g = a$.

To discretize (37), we approximate the solution $u = u_0 + \tilde{g}_D$ as $u \approx \Pi u_{\theta} + \tilde{g}_D$, where u_{θ} is a neural network, Π is the interpolation operator onto the non-Dirichlet degrees of freedom of the FE space, and an approximate lifting $\tilde{g}_D$ is obtained by interpolating g_D on the boundary degrees of freedom using standard FE procedures. We then minimize the loss $L(\theta) = E(\Pi u_{\theta})$.

Computational details We use a fully connected neural network u_θ with tanh activation and two hidden layers of width 32. We run NGD-based methods for 500 iterations and BFGS for 1.2×10^5 iterations. Since the loss can become negative, for NGD and NGD with CG we cannot employ the heuristic $\mu = \min(L(\theta), 10^{-5})$ as suggested in [72], and therefore we set $\mu = 10^{-9}$. In NyströmNGD we take $\mu = \max(p \cdot \hat{\lambda}_1 \cdot \epsilon_{\text{mach}}, 10^{-9})$.

Results On this test case, NyströmNGD and BFGS achieve comparable final accuracy, as shown in Figure 14, but NyströmNGD requires a fraction of the iterations and is approximately $10\times$ faster in wall-clock time. This efficiency arises from the excellent low-rank approximability of the Gramian, evident in Figure 15, which leads the Nyström rank parameter ℓ to stabilize around 80, far below the parametric dimension p . NGD and NGD with CG are not competitive in either iteration count or runtime, and both plateau at lower accuracy.

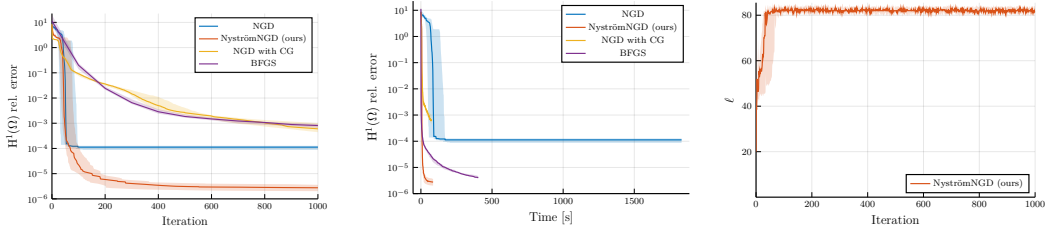


Figure 14: Poisson equation in 2D discretized via Deep Ritz and finite element interpolation. From left to right: relative $H^1(\Omega)$ error vs. iterations (first 1000 iterations), relative $H^1(\Omega)$ error vs. wall-clock time, and Nyström rank parameter ℓ vs. iterations, with $\ell_{\text{max}} = 500$.

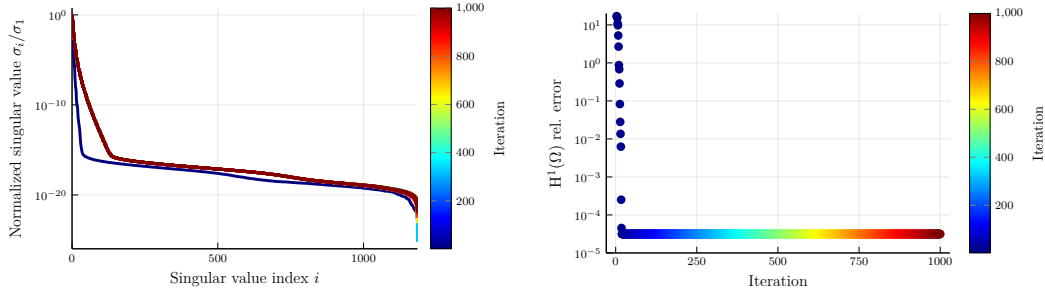


Figure 15: Poisson equation in 2D discretized via Deep Ritz and finite element interpolation, trained with NGD using an SVD-based pseudoinverse. On the left, the decay of the normalized singular values $\{\sigma_i/\sigma_1\}_{i=1}^p$ of the Gramian across iterations, with each color representing a different iteration. On the right, the relative $H^1(\Omega)$ error vs. iterations.

F.2 Diffusion-reaction-transport problem in 2D

We consider the diffusion-reaction-transport problem

$$\begin{cases} -\nabla \cdot (\kappa \nabla u) + \beta \cdot \nabla u + \sigma u = F & \Omega = (0, 1)^2, \\ u = g & \Gamma_D \subset \partial\Omega, \\ \kappa \nabla u \cdot \mathbf{n} = \psi & \Gamma_N \subset \partial\Omega, \end{cases} \quad (40)$$

where $\kappa(x, y) = 2 + \sin(x + 2y)$, $\beta(x, y) = [\sqrt{x - y^2 + 5} \quad \sqrt{y - x^2 + 5}]^\top$, and $\sigma(x, y) = e^{\frac{x}{2} - \frac{y}{3}} + 2$. The source term F is chosen so that the exact solution is

$$u(x, y) = \sin(3.2x(x - y)) \cos(4.3y + x) + \sin(4.6(x + 2y)) \cos(2.6(y - 2x)),$$

as in [7, 10]. The Dirichlet boundary Γ_D consists of the left and right sides of the domain, while the Neumann boundary Γ_N corresponds to the top and bottom edges.

The function spaces are $V = W = H_{0,\Gamma_D}^1(\Omega)$, and the variational formulation of (40) is defined as $\mathcal{A}u = f$ where

$$\begin{aligned}\langle \mathcal{A}u, v \rangle_{H_{\Gamma_D}^{-1} \times H_{0,\Gamma_D}^1} &= a(u, v) = \int_{\Omega} (\kappa \nabla u) \cdot \nabla v + (\beta \cdot \nabla u) v + \sigma u v \, d\mathbf{x} \\ \langle f, v \rangle_{H_{\Gamma_D}^{-1} \times H_{0,\Gamma_D}^1} &= \int_{\Omega} F v \, d\mathbf{x} + \int_{\Gamma_N} \psi v \, ds\end{aligned}$$

Since $\mathcal{A}$ is in general not symmetric positive definite, we adopt the least-squares formulation

$$\min_{u \in H_{0,\Gamma_D}^1(\Omega)} E(u) := \frac{1}{2} \|\mathcal{A}u - f\|_{H_{\Gamma_D}^{-1}}^2 = \langle \mathcal{A}u - f, \mathcal{R}_V^{-1}(\mathcal{A}u - f) \rangle_{H_{\Gamma_D}^{-1} \times H_{0,\Gamma_D}^1}, \quad (41)$$

with associated metric $g(v, w) := \langle \mathcal{A}v, \mathcal{R}_V^{-1} \mathcal{A}w \rangle$. Here, $\mathcal{R}_V^{-1} = (-\Delta)^{-1} : H_{\Gamma_D}^{-1} \rightarrow H_{0,\Gamma_D}^1$ denotes the inverse Riesz operator, which involves solving a Poisson equation.

To discretize (41) and the corresponding metric, we interpolate the neural network u_{θ} onto a conforming finite element space $V_h \subset V$, following [7], and minimize $L(\theta) = E(u_{\theta})$. Applying the discrete Riesz operator $\mathcal{R}_{V_h}^{-1}$ corresponds to solving a Poisson problem with FEM. To reduce computational cost, one could instead use a spectrally equivalent approximation, such as a few cycles of a geometric multigrid preconditioner. However, for simplicity, we solve the Poisson problem directly using FEM, computing a Cholesky factorization of the Laplacian matrix once.

Computational details We use a fully connected neural network u_{θ} with tanh activation and two hidden layers of width 32. We run NGD-based methods for 1000 iterations and BFGS for 1.2×10^5 iterations. For NGD and NGD with CG we use $\mu = \min(10^{-5}, L(\theta))$ as suggested in [72]. For NyströmNGD, to enforce stronger regularization in early iterations, we take $\mu = \max(p \cdot \hat{\lambda}_1 \cdot \epsilon_{\text{mach}}, 10^{-8} \cdot L(\theta)^2)$.

Results As illustrated in Figure 16, NyströmNGD reaches convergence in few iterations and achieves better accuracy compared to other NGD methods. The gap in convergence speed relative to BFGS is substantial in terms of both iterations and time. The smaller difference in wall-clock time between NyströmNGD and BFGS, compared to the previous variational example, can be attributed to the higher Nyström rank parameter ℓ at which NyströmNGD stabilizes. Although ℓ remains much lower than the parametric dimension p , it is higher than in earlier examples. This is due to the slower, yet still strong, decay of the singular values of the Gramian, as shown in Figure 17.

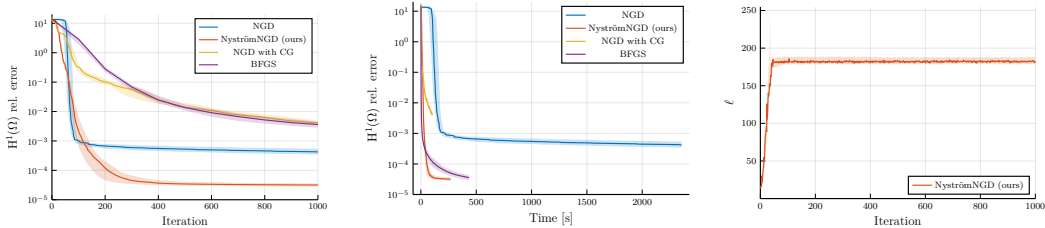


Figure 16: Diffusion-reaction-transport equation in 2D discretized via Finite Element Interpolated Neural Networks. From left to right: relative $H^1(\Omega)$ error vs. iterations (first 1000 iterations), relative $H^1(\Omega)$ error vs. wall-clock time, and Nyström rank parameter ℓ vs. iterations, with $\ell_{\text{max}} = 500$.

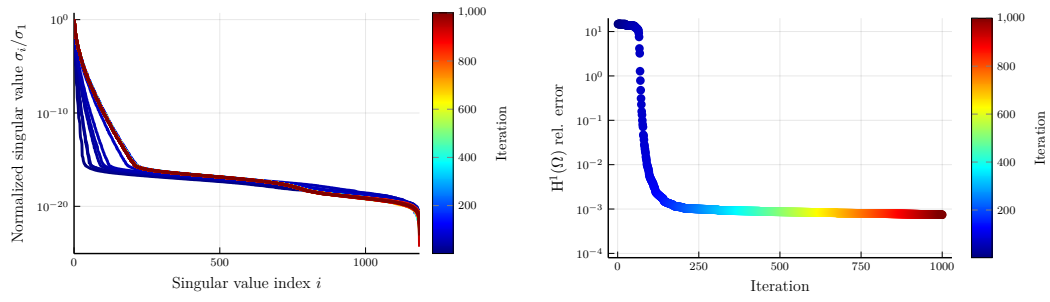


Figure 17: Diffusion-reaction-transport equation in 2D discretized via Finite Element Interpolated Neural Networks, trained with NGD using an SVD-based pseudoinverse. On the left, the decay of the normalized singular values $\{\sigma_i/\sigma_1\}_{i=1}^p$ of the Gramian across iterations, with each color representing a different iteration. On the right, the relative $H^1(\Omega)$ error vs. iterations.